LNA Website Development Blueprint

Nov 3, 2026 · @Refat

This blueprint plans the rebuild of lnabd.org as a Next.js, Supabase and SSLCOMMERZ system built in Google Antigravity, with centralized JSON content behind a swappable repository layer. Part 2 lists the SRS conflicts and gaps that need your decision before coding starts.

Part 1: Executive project understanding

LNA (Lead Nation Academy) needs a public site, a learner portal and an admin portal that take offline-course enrollments, collect BDT payments through SSLCOMMERZ, and issue receipts and certificates. The SRS (LNA-SRS-001, 28 Oct 2026) is the functional authority; this blueprint adds the architecture, security design and Antigravity workflow it does not contain.

What I could and could not inspect

- Read in full: your master prompt and all 48 pages of the SRS, including the page concepts in Appendix A.
- Fetched as text: lnabd.org home, /about, /community and the IRGP course page. I did not see the live site rendered, and the fetch returns no stylesheet, so **exact brand hex values and fonts are unverified**. Part 4 handles this with a one-step extraction gate.
- Not inspected: /news, /terms-and-conditions, /privacy-policy, the live contact form behavior, and any staging or admin area.

Goals

1. Replace the Elementor/WordPress site with a maintainable Next.js system that satisfies every SRS requirement ID (Appendix A traces all of them).
2. Keep the established identity: logo, palette and academic tone, with accessibility-driven adjustments documented.
3. Make ordinary content edits possible in centralized JSON during development, and in admin forms after launch, without touching components.
4. Treat payments as a production financial system from the first commit.
5. Build through small, verifiable Antigravity tasks so no session redesigns the architecture.

Constraints taken from the SRS

- Offline courses only: no LMS features. Paid enrollment via SSLCOMMERZ hosted checkout; refunds are approved online but transferred manually.

- About 100,000 visits a month with bursts near 500 concurrent readers, so public pages must be cached and the database touched only by transactions and portals.
- Staff need a second factor; public signup and Google login are off; applicants have no Auth account until payment is validated.
- English editorial content; the language switch translates interface labels only.

Assumptions I made (each needs a yes or no from you)

#	Assumption	Why	Confirm with
A1	Currency is BDT only, stored as integer poisha (1 BDT = 100)	SSLCOMMERZ settles in BDT; avoids float errors (SRS 8.2)	You / finance
A2	Next.js 16 line, Supabase, SSLCOMMERZ and an email provider are fixed choices	Named in SRS Figures 1, 4 and 7 and Section 9	You
A3	One production environment plus one staging environment, each with its own Supabase project	SRS 13 requires isolated staging with sandbox payments	You
A4	Certificate names are Latin script in v1	PDF text shaping for Bangla is a real risk (Part 12)	LNA operations
A5	A paid-but-seat-lost payment goes to staff review and manual refund	SRS is silent on this case (Part 2, gap G6)	LNA finance
A6	Hosting on a paid commercial tier, not a free hobby tier	Commercial use of a free tier may breach its terms; verify at purchase	You

Part 2: Existing website and SRS analysis

The live site is a thin Elementor marketing page with placeholder content and no visible transaction system, so the SRS is effectively a new product. The bigger risk is not the rebuild but content that looks verified on the live site and is not. Sixteen gaps and conflicts below need a decision; items marked Blocks stop a specific phase.

2.1 What the live site shows

Area	Observed on lnabd.org (fetched as text, Nov 2026)	Implication
Platform	WordPress with Elementor 4.3.5 (page meta)	Full replacement; no data to migrate except content and URLs
Navigation	Courses, Internship, Community, News, About, plus an ENROLL NOW button	SRS nav is Home, About, Courses, Events, Mentors, Contact, Login (gap G1)
Home hero	Two slides: Skill-Based Certifications; Internship Opportunity (sic)	SRS wants three managed slides: course, assessment, event
Home body	Lorem ipsum under "From Classroom to Boardroom"; BOOK NOW links to #	Category B placeholder; no impact figures exist yet
Partners	Strip of seven logos including BTRC and the ICT Division	Affiliation claim; needs written permission before reuse (G3)
Courses	One program (IRGP) shown as two cards with different class counts	Data conflict (G3)
Course page	Fee BDT 9,999 against BDT 20,000 ("Save 50%"), batch starts 12 Dec, 16 modules, Fridays, venue AIUB ICE Banani, curriculum for 12 classes, FAQ	Good source for the first real course record
Mentors	12 named mentors with titles and the modules they teach	Real people: needs consent and verified titles (G3)
Testimonials	Lorem ipsum, one name repeated, stock photo	Category B; never publish as real (FR-REV-001)
Internships	Five company cards with on-site schedule and monthly stipend ranges	Not in the SRS (G1)
Community	"Coming soon"	Not in the SRS
Contact	Form labelled "Contact Form Demo"; Tejgaon address, info@lnabd.org, +8801581799666	Candidate Category A, to confirm
Footer	Copyright 2025; third-party developer credit; Facebook and LinkedIn links	Update year and credit

Area	Observed on lnabd.org (fetched as text, Nov 2026)	Implication
Brand assets	Logo file LNA-BR-Logo.png in WordPress uploads	Reuse; extract palette and fonts in Phase 1 (Part 4)

2.2 What the SRS defines (extracted)

Item	Count or content
Functional requirements	77: PUB 7, CMS 5, CRS 7, ENR 6, PAY 9, AUT 7, LRN 6, RFC 8, EVT 5, MEN 1, REV 1, OTH 9, ADM 6
Non-functional requirements	15 (performance, capacity, availability, security, privacy, accessibility, SEO, compatibility, backup, monitoring, maintainability)
Business rules	18 (BR-01 to BR-18)
Use cases and acceptance tests	4 use cases (UC-01 to UC-04); 15 end-to-end tests (AT-01 to AT-15)
Roles	Visitor, Applicant (no account), Learner, Admin, Super Admin
Pages	13 public, 8 enrollment, 5 account, 8 learner, 18 admin, 5 system (SRS 4.2)
Entities	14 in SRS 8.1; Part 6 adds supporting tables
Integrations	Supabase (Postgres, Auth), SSLCOMMERZ hosted checkout, transactional email, privacy analytics plus consent-gated GA4
Priorities	All M except FR-PUB-006, FR-EVT-003 to 005, FR-MEN-001, FR-REV-001 (G)

2.3 Gaps, conflicts and decisions

ID	Finding	Recommended resolution	Needs your decision
G1	The live site has Internship, Community and News; the SRS excludes them (News is a future option, internships are absent)	Keep the SRS navigation. Carry internship content as a section inside the IRGP course page and the About page. 301-redirect /news and /community to the home page until built	Yes
G2	The course URL /industry-readiness-graduate-program-irgp/ has SEO value	Keep the slug: new route /courses/industry-readiness-graduate-program-irgp with a 301 from the old path	No
G3	The live copy contradicts itself: 16 or 14 classes, 1.5 to 2 hours or 3 hours, one or two days a week, 8 weeks or 2 months, fee 9,999 or 20,000; two mentors have different employers on different pages; two contact emails (info@lnabd.org, leadnationacademy@gmail.com); the FAQ claims employer-endorsed certification	Nothing from the live site is Category A until you confirm it in writing. Seed data marks every disputed field verified:false	Yes, blocks Phase 4 content
G4	Free batches confirm and activate an account (FR-ENR-005, AT-05) without email proof first, so fake emails can consume seats	Apply Turnstile and rate limits; hold the seat for 30 minutes; confirm only after the emailed setup link is opened	Yes
G5	Staff second factor allows an emailed code (FR-AUT-004). An inbox takeover then breaks both password reset and the second factor. Supabase native MFA covers authenticator apps and phone codes, not email codes	Build the email code as a custom, hashed, single-use flow. Require an authenticator app for Super Admin; allow email code for Admin only	Yes, deviates from SRS wording

ID	Finding	Recommended resolution	Needs your decision
G6	The SRS does not say what happens when a delayed payment is validated after the seat hold expired and the batch is full	Always record the payment. If a seat exists, confirm; if not, set enrollment to review, alert staff, refund manually (assumption A5)	Yes
G7	Figure 4 and BR-02 say only the validated gateway notification marks payment paid, yet the buyer lands on a success page	Define "validated notification" as IPN or success-redirect data checked against the Order Validation API server-side. Add a scheduled reconciliation by transaction ID	Yes, clarify wording
G8	SRS mockups show sequential references (LNA-2026-01842, LNA-EVT-00842) but SRS 8.2 forbids sequential public references	Random Crockford base32 references, for example LNA-7K3M-9XQ2, unique-constrained	No
G9	FR-PAY-001 lets staff configure universal discounts; BR-09 limits them to Super Admin	Universal discount: Super Admin only. Batch discount: Admin	Yes
G10	FR-RFC-001 depends on a "policy period" that no document defines	Store refund policy as structured fields per course (days after payment, days before start) plus text	Yes, blocks Phase 11
G11	The cover says version 1.0 and every page footer says 2.0; approval record is blank	Fix before sign-off so change control has a baseline	Yes
G12	FR-PUB-002 (M) requires reviews and mentors on the home page, but FR-PUB-006, FR-MEN-001 and FR-REV-001 are G priority	Build read-only mentor and review sections in the public phase; build their admin forms later	No

ID	Finding	Recommended resolution	Needs your decision
G13	Personal data will sit in a cloud database; Bangladesh's Personal Data Protection Ordinance 2025 limits cross-border transfer and ties enforcement to the National Data Governance Authority	Choose the Supabase region only after legal advice; record the decision (Part 7, section 7.9)	Yes, blocks production
G14	Receipts are required (FR-PAY-009, FR-LRN-003) but no format, numbering or tax treatment is given	Sequential internal receipt number separate from public reference; PDF; confirm VAT treatment with LNA's accountant	Yes
G15	FR-ENR-003 says "prevent or reconcile" a duplicate application	Reuse the existing pending enrollment for the same email and batch and create a fresh payment attempt	No
G16	Bangla copy, assessment questions, certificate artwork and policy text are not supplied (SRS 14)	Use labelled demo content now; Phase gates name each missing input	Yes, inputs

Part 3: Recommended technology stack

Use Next.js (App Router) with TypeScript, Tailwind CSS 4 and Supabase, deployed on a paid Vercel plan, with Resend for email and a thin in-house SSLCOMMERZ client. The SRS already names Next.js, Supabase and SSLCOMMERZ; I tested each against alternatives rather than assuming them, and I recommend one change: do not integrate bKash directly (below).

3.1 Decision matrix

Layer	Choice	Alternatives weighed	Why this one	Main risk and mitigation	Version status
Runtime	Node.js 24 LTS, pinned in .nvmrc and engines	Node 22 LTS; a newer LTS if released	Node's schedule lists 24 as LTS to April 2028 and 22 to April 2027	Node 24 enters maintenance on 20 Oct 2026 per the schedule: check nodejs.org for a newer Active LTS and take it if Next 16 supports it	Verify at install
Framework	Next.js 16 line, App Router	Astro plus separate API; Remix; plain React SPA	One codebase serves cached public pages, protected portals and gateway callbacks; matches SRS Figure 4	Cache and middleware semantics changed in 15 and 16: isolate revalidation in one module; never rely on middleware alone for authorization	Sources disagree: 16.2.7 (June 2026), 16.4.1 (15 Oct 2026, one aggregator). Pin the latest 16.x from the npm registry at install. Treat any Next 17 claim as unverified
Language	TypeScript, strict mode	JavaScript	Schema-to-type safety for the JSON and API layers	Antigravity may weaken types: lint rule bans any and ts-ignore	Latest stable, not a pre-release
Styling	Tailwind CSS 4.x with design tokens in CSS variables	CSS Modules; Bootstrap	Token-driven, fast, consistent for AI-generated components	v4 replaced tailwind.config.js with CSS config: block v3 snippets in the rules	4.3.1 (12 Jun 2026) and 4.3.3 (Jul 2026) reported by secondary sources; verify
Accessible primitives	Radix UI primitives with own styling	shadcn/ui as a package set; Headless UI	Dialogs, menus, tabs and focus management are the usual WCAG failures	Many small packages: allowlist only the ones used	Verify

Layer	Choice	Alternatives weighed	Why this one	Main risk and mitigation	Version status
Data and auth	Supabase: Postgres, Auth, Storage	Firebase; self-hosted Postgres with custom auth; Neon plus Auth.js	RLS, MFA (TOTP) and storage in one managed service; SRS 9.2 requires it	Vendor lock-in and cross-border hosting (G13): keep SQL migrations portable; no Supabase-only logic outside RPC functions	supabase-js 2.x and @supabase/ssr; verify exact versions
Data access	Generated types from the Supabase CLI; reads through supabase-js under RLS; critical writes through Postgres functions (RPC)	Prisma or Drizzle ORM	Capacity and payment confirmation must be single transactions with row locks; functions give that, and an ORM adds a layer	Logic in SQL needs tests: pgTAP suite in CI	Supabase CLI: verify
Validation	Zod, one schema per entity, shared by JSON loader, forms and route handlers	Valibot; hand-written checks	Same schema validates content files, form input and API bodies	Major-version API drift: pin	Verify
Payments	SSLCOMMERZ hosted checkout, own typed client using fetch	Unofficial npm wrappers; bKash direct; Stripe	Gateway required by SRS; the wrappers I found are third-party and unaudited	Gateway doc quirks: client is about 150 lines, fully tested against sandbox	API v4 docs opened 3 Nov 2026
Rate limiting and bots	Upstash Redis rate limit plus Cloudflare Turnstile	Postgres counters; Vercel firewall rules only	Cheap, serverless-friendly, covers login, status lookup, forms, payment initiation (NFR-SEC-03)	Two extra vendors: both optional behind interfaces	Verify

Layer	Choice	Alternatives weighed	Why this one	Main risk and mitigation	Version status
Email	Resend with a provider interface	Amazon SES; Postmark	Simple API, domain verification, delivery webhooks	Inbox placement in Bangladesh: SPF, DKIM, DMARC and a staging test to Gmail and local providers	Verify plan limits
Hosting	Vercel (paid commercial plan), functions in the region nearest the database	Cloudflare; a VPS with Docker	First-class Next.js support, preview deployments, CDN	Free tier is non-commercial (A6); nearest edge to Dhaka is not local: measure LCP from Dhaka before launch	Verify plan terms at purchase
Scheduled jobs	Postgres pg_cron for hold expiry; authenticated Vercel Cron routes for reconciliation and email retries	A queue service	No extra vendor	Cron limits per plan: correctness never depends on a job (holds are also checked at read time)	Verify
Certificates and receipts	pdf-lib with embedded fonts over an A4 landscape template	Headless Chromium; react-pdf	Fixed field positions (FR-RFC-007) suit overlay on artwork; small runtime	Bangla shaping unsupported: Latin names only in v1 (A4)	Verify
Carousel	Embla Carousel, own controls	Custom code; Swiper	Swipe plus keyboard, pause and reduced motion (FR-PUB-004) built on top	Auto-advance must honor prefers-reduced-motion	Verify
Internationalization	next-intl with /bn path prefix	Cookie-based switch	A cookie read forces dynamic rendering and breaks CDN caching; path prefix keeps both languages static	Duplicate English content on /bn: canonical to English, noindex on /bn	Verify Next 16 support

Layer	Choice	Alternatives weighed	Why this one	Main risk and mitigation	Version status
Monitoring	Sentry with PII scrubbing; uptime checks; Vercel logs	Datadog	Covers NFR-OBS-01 at low cost	Leaking personal data in events: scrub in beforeSend	Verify
Analytics	Plausible or Umami, plus GA4 behind consent	GA4 only	SRS 9.4 requires privacy-friendly plus consent-aware	Consent banner wiring: test AT-14	Choose one at Phase 12
Testing	Vitest, Testing Library, Playwright with axe-core, pgTAP, k6, Lighthouse CI	Jest; Cypress	Covered in Part 11	None notable	Verify
CI/CD	GitHub Actions plus Vercel preview and production deploys	GitLab CI	Required gates in Part 11	Secrets in CI: environment-scoped, never in logs	Not versioned
Package manager	pnpm with exact pins and a committed lockfile	npm	Strict dependency isolation stops phantom imports; frozen installs in CI	Node 24 may not ship Corepack: install pnpm explicitly and record the version	Verify

3.2 bKash

The SRS needs one gateway. SSLCOMMERZ hosted checkout normally lists mobile-banking wallets, so confirm at merchant onboarding that bKash and Nagad appear on the checkout page. A second direct integration doubles the reconciliation and security surface for no SRS requirement, so exclude it.

3.3 Dependency discipline

1. Resolve exact versions once, in Session 1 of Part 13: run `pnpm view <pkg> version` for each allowlisted package, confirm against the official docs page, then commit the versions and the lockfile.

2. Save exact versions only (`save-exact=true` in .npmrc). CI installs with `--frozen-lockfile`.
3. The allowlist lives in docs/DEPENDENCIES.md with one line of purpose per package. Antigravity may not add or upgrade anything outside it without a written request in the task.
4. Upgrades happen as a separate pull request: Renovate grouped weekly, no auto-merge for next, react, typescript, tailwindcss, supabase, zod.
5. Run `pnpm audit` and an OSV scan on every pull request. Block merges on high or critical advisories.
6. Install scripts are off by default for new packages; review any package that needs them.

Initial allowlist

Group	Packages
Core	next, react, react-dom, typescript, tailwindcss, @tailwindcss/postcss
Data and auth	@supabase/supabase-js, @supabase/ssr, zod
UI	@radix-ui primitives actually used, embla-carousel-react, lucide-react, clsx, tailwind-merge
i18n	next-intl
Services	resend, @upstash/ratelimit, @upstash/redis, @sentry/nextjs, pdf-lib, @pdf-lib/fontkit
Dev and test	vitest, @testing-library/react, jsdom, @playwright/test, @axe-core/playwright, eslint, eslint-plugin-jsx-a11y, prettier, supabase (CLI), @lhci/cli
External binaries in CI	gitleaks, k6, OWASP ZAP baseline, OSV scanner

Part 4: UI/UX and design-system blueprint

The design direction is a calm, credential-focused academic look: generous whitespace, one primary action per screen, real mentor photography, and plain labels. Brand colors come from a one-time extraction from the live site and then live only in `brand.json`; a generator writes the CSS tokens and a test fails the build if any pair misses WCAG contrast.

4.1 Brand identity: preserve first, adjust only with a record

I could not read the live site's stylesheet, so no hex value in this document is verified. Gate B0 closes that:

1. Open lnabd.org in a desktop browser, then read the Elementor Global Colors and Global Fonts (Site Settings), or the computed styles of the header, primary button, footer and headings in DevTools.
2. Record every color, font family and weight in docs/brand-extraction.md with the page element it came from and a screenshot.
3. Ask LNA for the logo as SVG; until then use the live LNA-BR-Logo.png from the site's uploads folder.
4. Fill `content/config/brand.json` from that file. Nothing else defines a color.

The SRS page concepts show the intended feel without fixing values: dark navy for headers and primary surfaces, teal for primary actions and positive states, gold or amber for secondary calls to action and discount badges, a muted red for refusals and destructive actions, and a pale blue-grey page background. Use these as role guidance only.

Adjustment policy. If a brand color fails contrast for its role, keep the original for decoration and graphics, and add a darker or lighter `-text` or `-on` variant in the same hue for text and controls. Record each change and the ratio in docs/brand-decisions.md. Do not change a primary brand color without that record.

4.2 Tokens

Token group	Names (values come from brand.json)	Rule
Brand	primary, primary-on, accent, accent-on	Primary carries main actions; accent only for secondary calls to action and discount badges
Surface	page, card, card-raised, sidebar, footer	Admin sidebar and footer use the darkest brand tone
Text	text, text-muted, text-inverse, link	Body text at least 4.5:1 on its surface; muted text still 4.5:1
Status	success, warning, danger, info, neutral (each with -bg, -text, -border)	Always paired with a text label, never color alone (SRS 11.1)

Token group	Names (values come from brand.json)	Rule
Focus	focus-ring (two-tone)	Visible on every surface, never clipped
Shape and space	radius-sm/md/lg, space scale 4 px base, shadow-1/2	Cards 12 px radius, controls 8 px
Motion	duration-fast/base, easing	All motion disabled under prefers-reduced-motion

Generator and gate. `pnpm tokens` reads brand.json, writes src/styles/tokens.css (CSS variables consumed by Tailwind's theme), and a Vitest test computes contrast for every declared text-on-surface pair. A failing pair fails CI.

4.3 Typography

- Keep the live site's font family if extraction finds one that is open licensed; otherwise use a humanist sans for Latin text.
- Add Noto Sans Bengali (or another open Bangla face LNA approves) for the translated interface labels (FR-OTH-007).
- Self-host through next/font to avoid third-party requests and layout shift.
- Scale: body 16 px minimum, line height 1.5, headings on a 1.25 ratio, line length 65 to 75 characters. Text must stay usable at 200 percent zoom.

4.4 Academic credibility rules (design and content)

- No statistic, accreditation, endorsement or partner logo appears unless its record has verified:true and an evidence note (SRS FR-PUB-005; your rule on Category B).
- Mentors show real photos and titles only with consent on file.
- Demo content is visibly marked in development with a corner ribbon driven by the content's classification, and the production build fails if any rendered record is demo.
- Prefer text over decoration: no stock testimonials, no animated counters, no parallax.

4.5 Component inventory

Group	Components
Layout	SiteHeader, MobileNav, SiteFooter, Container, SectionHeader, Breadcrumb, LanguageSwitch

Group	Components
Public content	HeroSlider, AssessmentInvite, ImpactStats, CourseCard, BatchCard, FeeSummary, EventCard, MentorCard, ReviewCarousel, CurriculumAccordion, FaqAccordion, PartnerStrip
Forms	Field, TextInput, Select, Checkbox, ConsentCheckbox, ErrorSummary, SubmitButton (pending state), OtpInput, TurnstileWidget
Feedback	StatusBadge, Alert, EmptyState, ClosedNotice (replaces Apply on closed or full items)
Data	DataTable (prioritized columns on narrow screens), FilterBar, Pagination, ConfirmDialog (names the affected record)
Portals	PortalShell, Sidebar, StatCard, AnnouncementCard, ReceiptCard, CertificateCard
Admin editing	EditorForm, DraftPublishBar (Save Draft, Preview, Publish, Archive), DependencyWarning

Each component lives once under src/components. A new component may be created only after a search for an existing one (Part 8, rule R6).

4.6 Responsive behavior

Use the SRS breakpoints exactly: 320 to 599 px single column with compact navigation and no horizontal table scrolling in learner flows; 600 to 1023 px two-column cards and a collapsible admin menu; 1024 px and above full navigation and a fixed admin sidebar. Configure Tailwind screens as sm 600 px and lg 1024 px. Playwright runs every core journey at 320, 768 and 1280 px.

4.7 Page patterns (from SRS Appendix A)

Page	Pattern and rules
Home (A01)	Eight sections in SRS 4.1 order; hero slider with arrows, indicators, a visible pause control, swipe and keyboard; no autoplay under reduced motion

Page	Pattern and rules
Courses (A02)	Open, Upcoming and Closed groups as tabs or filter chips; no text search (FR-CRS-003); course page shows batch, seats left, original fee, discount and final fee before Enroll
Enrollment (A03)	Two steps: universal form, then fee review; one primary button; consent checkbox links to Privacy and Refund Policy; entries preserved on error
Payment result and status (A04)	Result page shows pending until the server confirms; status lookup asks reference and email, returns status and next step only
Assessment (A05)	One question per screen with progress, then name, email and consent, then result by email
Events (A06)	Upcoming and Previous toggle; free registration form; email pass
Mentors and contact (A07)	Profile cards; protected inquiry form with Turnstile
Login (A08)	Email and password; staff verification step with authenticator or email code; paste and password managers allowed
Learner portal (A09, A10)	Overview cards, course schedule and venue, receipt, refund request, certificate; no LMS elements
Admin (A11 to A14)	Structured forms, list views with search, filter and pagination, attention queue, CSV export

4.8 WCAG 2.2 AA checklist built into the components

Criterion	Component rule
1.4.3, 1.4.11 contrast	Enforced by the token test: text 4.5:1, controls and focus 3:1
2.1.1, 2.4.3, 2.4.7 keyboard and focus	Every control reachable and visibly focused; dialogs trap and restore focus
2.4.11 focus not obscured	Sticky header never covers a focused element: use scroll-margin on anchors and form errors

Criterion	Component rule
2.5.7 dragging	Carousels offer arrows and indicators besides swipe
2.5.8 target size	Interactive targets at least 24 by 24 CSS px, 44 px on touch layouts
3.2.6 consistent help	Support contact appears in the same footer position on every page
3.3.1, 3.3.3 error identification	Message beside the field plus an ErrorSummary on long forms
3.3.7 redundant entry	Enrollment never asks twice for the same data; learner profile prefills
3.3.8 accessible authentication	No cognitive tests; password and code fields accept paste and autofill (autocomplete one-time-code)
2.2.2, 2.3.3 motion	Pause control and reduced-motion support on the hero
1.4.4 resize	Layouts hold at 200 percent zoom and 320 px width

Automated axe scans cover part of this; the remaining items need the manual audit in Part 11 (AT-15).

Part 5: JSON-driven content architecture

Organize JSON by domain, validate every file with Zod at build time, and read it only through a repository interface. This resolves a conflict in the requirements: the SRS makes admins edit courses, batches, events, mentors, reviews and contact settings through forms with no deployment (FR-CMS-001, FR-CMS-002), and a deployed site cannot write its own JSON files. So JSON does three jobs: permanent site configuration, demo data during development, and seed data for the database. At launch, admin-managed content moves to Supabase behind the same interface and no component changes.

5.1 Where each kind of content lives

Content	Development (Phases 3 to 8)	Production	Changed by
Organization name, legal name, logo paths	JSON	JSON	Edit file, redeploy
Brand colors and fonts	JSON, then `pnpm tokens`	JSON	Edit file, redeploy
Navigation, footer links, social links	JSON	JSON	Edit file, redeploy
SEO defaults and templates	JSON	JSON	Edit file, redeploy
Interface labels, English and Bangla	JSON (messages)	JSON	Edit file, redeploy
Privacy, Terms, Refund Policy text	Markdown files	Markdown files, versioned in Git	Edit file, redeploy
Courses, batches, discounts	JSON demo	Database	Admin form (discounts for pricing always from the database)
Events, mentors, reviews, hero slides, impact figures	JSON demo	Database	Admin form
Contact details	JSON	Database settings, JSON as seed	Admin form
Assessment questions and scores	JSON seed	Database (versioned)	Admin or migration
Enrollments, payments, accounts, refunds, certificates, audit	Never JSON	Database	Application code only

Category C data (operational records) is never in JSON. The validator rejects it by schema and by a key-name scan.

5.2 Classification: real, demo, dynamic

Every record carries a `provenance` block, so classification is per record and not per file:

```
"provenance": {
  "class": "B",
  "verified": false,
  "source": "Copied from lnabd.org on 2026-11-03; fee and class count
conflict between pages",
  "verifiedBy": null,
  "verifiedOn": null
}
```

- Class A: verified, `verified:true`, with `source`, `verifiedBy` and `verifiedOn` all set.
- Class B: demo or unconfirmed. Rendered with a visible "Demo content" ribbon in development and preview.
- Production build gate: `NODE_ENV=production` plus `CONTENT_STRICT=true` makes the loader throw if any rendered record is class B. A claim such as accreditation, an impact figure or a testimonial cannot ship unverified.
- Class C does not exist in JSON by design.

5.3 Directory structure

By domain rather than by page or one file per entity type: a domain changes together, has one owner, and keeps references local.

```
content/                      # never inside /public, so raw files are not
downloadable
  site/
    organization.json         # names, address, phone, email, hours
    brand.json                # colors, fonts, logo paths, favicon
    navigation.json           # header and footer link structure
    social.json
    seo.json                  # defaults, title templates, structured-data
fields
  home/
    hero-slides.json          # 3 slides: course, assessment, event
    sections.json             # section order and toggles (SRS 4.1)
    impact-figures.json       # only verified figures render
  catalog/
    courses/irgp.json         # one file per course
    batches/irgp-batch-1.json # one file per batch, references course slug
    discounts.json            # demo only; production prices come from the
database
    mentors/ilmul-haque-sajib.json
  events/
    sample-networking-evening.json
  social-proof/
```

```
    reviews.json              # consent fields required
  assessment/
    questions.v1.json         # 15 questions, option scores 0 to 4
  messages/
    en.json
    bn.json                   # interface labels only (BR-17)
  policies/
    privacy.en.md
    terms.en.md
    refund.en.md
  schema/                     # Zod schemas, also exported as JSON Schema for
editors
src/lib/content/
  repository.ts               # interfaces (CourseRepository and others)
  json/                       # JsonContentRepository (development, seed,
tests)
  supabase/                   # SupabaseContentRepository (production)
  index.ts                    # picks implementation from CONTENT_SOURCE per
domain
```

5.4 Sample schemas and records

Money is stored as integer poisha. Dates are ISO 8601 with offset. Slugs are unique and are the public URL.

```
// content/schema/course.ts (abridged)
export const Provenance = z.object({
  class: z.enum(["A", "B"]),
  verified: z.boolean(),
  source: z.string().min(5),
  verifiedBy: z.string().nullable(),
  verifiedOn: z.string().date().nullable(),
}).refine(p => p.class === "B" || (p.verified && p.verifiedBy &&
p.verifiedOn));

export const Course = z.object({
  slug: z.string().regex(/^[a-z0-9-]+$/),
  title: z.string().min(3),
  summary: z.string().max(240),
  overview: z.string(),
  eligibility: z.array(z.string()),
  outcomes: z.array(z.string()),
  topics: z.array(z.object({ title: z.string(), objectives:
z.array(z.string()) })),
  mentorSlugs: z.array(z.string()),
  faq: z.array(z.object({ q: z.string(), a: z.string() })),
```

```
  refundPolicy: z.object({ daysAfterPayment:
z.number().int().nonnegative().nullable(),
                           daysBeforeStart:
z.number().int().nonnegative().nullable(),
                           text: z.string() }),
  cover: z.object({ src: z.string(), alt: z.string().min(5) }),
  status: z.enum(["draft", "published", "archived"]),
  provenance: Provenance,
}).strict();   // unknown keys, such as a password, fail validation
```

```
{
  "slug": "industry-readiness-graduate-program-irgp",
  "title": "Industry Readiness Graduate Program (IRGP)",
  "summary": "Offline program that prepares students and fresh graduates for
corporate roles.",
  "mentorSlugs": ["ilmul-haque-sajib"],
  "refundPolicy": { "daysAfterPayment": null, "daysBeforeStart": null,
"text": "TO BE SUPPLIED BY LNA" },
  "status": "published",
  "provenance": { "class": "B", "verified": false, "source": "Copied from
lnabd.org 2026-11-03; unconfirmed", "verifiedBy": null, "verifiedOn": null }
}
```

```
{
  "slug": "irgp-batch-1",
  "courseSlug": "industry-readiness-graduate-program-irgp",
  "startsOn": "2026-12-12",
  "schedule": "DEMO: Fridays, 3:00 PM to 6:30 PM",
  "venue": "DEMO venue",
  "capacity": 30,
  "listFeeMinor": 999900,
  "enrollmentOpensAt": "2026-11-10T00:00:00+06:00",
  "enrollmentClosesAt": "2026-12-10T23:59:00+06:00",
  "status": "published",
  "provenance": { "class": "B", "verified": false, "source": "Fee 9,999 on
lnabd.org conflicts with the 20,000 list price", "verifiedBy": null,
"verifiedOn": null }
}
```

5.5 How the frontend reads it

```
// app/[locale]/courses/page.tsx  (server component, statically generated)
import { content } from "@/lib/content";
import { CourseCard } from "@/components/public/CourseCard";

export default async function CoursesPage() {
  const groups = await content.courses.listGroupedByEnrollmentState(new
Date());
  return <CourseGroups groups={groups} />;   // labels come from
t("courses.open"), never literals
}
```

Rules that keep components free of content: components receive typed props; any visible string is either record content or a message key; no component imports from `content/` directly, only through `@/lib/content`.

5.6 Validation strategy

- `pnpm content:validate` runs before `build` and in CI. It parses every file with its Zod schema, checks references (a batch's course exists, a mentor slug exists, a slide's target page exists), slug uniqueness, image files exist, alt text present, and English and Bangla message keys match.
- A secret scan rejects keys or values that look like credentials (password, secret, token, apikey, store_passwd, service_role).
- Failure stops the build. The site never serves unvalidated content.
- Payment and enrollment server code may not import the content module. An ESLint restriction enforces it: prices always come from the database (SRS FR-PAY-003).

5.7 Changing content versus changing functionality

Change	Where	Code change	Database migration	Redeploy
Edit a course title or summary in development	JSON	No	No	Yes
Edit a course after launch	Admin form	No	No	No
Change a navigation label or footer link	JSON	No	No	Yes
Add a Bangla label	messages/bn.json	No	No	Yes

Change	Where	Code change	Database migration	Redeploy
Change brand color	brand.json then `pnpm tokens`	No	No	Yes
Add a field to courses (for example language of instruction)	Schema, repository, form	Yes	Yes	Yes
Change how discounts combine or how a payment is validated	Code and database functions	Yes	Maybe	Yes

JSON changes presentation and data. It never holds business rules, permissions or prices used for charging.

5.8 Migration path to the database and a CMS

1. Every domain is accessed through a repository interface from the first day (Phase 3).
2. Phase 6 writes SQL migrations that mirror the Zod schemas and a `pnpm db:seed` script that loads the JSON through the same schemas, keeping each record's provenance.
3. `CONTENT_SOURCE_<DOMAIN>=json|supabase` (server-only) selects the implementation per domain, so catalog can move before events.
4. One contract test suite runs against both repositories and must pass identically.
5. Cutover per domain; publishing in the admin invalidates the relevant cache tags (SRS 4.3).
6. JSON stays as seed data, test fixtures and the demo environment. A headless CMS later means a third repository implementation and nothing else.

Part 6: Full system architecture

One Next.js application serves three surfaces (public, learner, admin). Public pages are cached; every state change goes through server-only services and Postgres functions that hold the transactions. Browsers never write to operational tables. This section fixes the boundaries; Part 7 secures them.

6.1 What lives where

Component	Responsibility	Never does
Browser	Renders pages, collects form input, shows server results	Decide price, payment status, role or ownership
Next.js on Vercel (CDN)	Serves cached public pages; runs server actions, route handlers and server components	Hold secrets in client bundles; trust browser-supplied amounts
Server modules (src/server)	Authorization, validation, business rules, gateway calls, email, PDFs, audit	Run without the server-only import guard
Supabase Postgres	Source of truth; constraints, row locks and RPC functions for capacity, payment confirmation, refunds, certificate numbers	Expose operational tables to the anon key
Supabase Auth	Passwords, sessions, TOTP; users created only by the server	Allow public signup
Supabase Storage	Public media bucket; private bucket for receipts, certificates and templates	Serve private files without a signed, ownership-checked URL
SSLCOMMERZ	Hosted checkout, IPN, Order Validation and transaction query APIs	Be trusted without server-side validation
Resend	Transactional email, delivery webhooks	Block a validated payment
Upstash Redis and Turnstile	Rate limits and bot checks	Store personal data
Sentry, uptime checks	Errors and health	Receive secrets or personal data
Analytics	Page views; GA4 only after consent	Receive names, emails, references or answers

6.2 Frontend architecture

Route groups: `(public)` under `[locale]` for en and bn, `(auth)` for login, setup and reset, `(learner)` for the portal, `(admin)` for staff, and `api` for callbacks.

Surface	Rendering	Caching	Auth
Home, About, Courses, Course, Events, Event, Mentors, Mentor, Policies, Contact, Assessment	Static with revalidation by tag	CDN, invalidated on publish	None
Enrollment steps, status lookup, free event form	Static shell, client form, server action	Not cached for submissions	None, Turnstile and rate limit
Payment result pages	Dynamic	no-store	Unguessable token
Learner portal	Dynamic server components	no-store	Learner session
Admin portal	Dynamic server components	no-store	Staff session plus verified second factor

The language switch uses a `/bn` path prefix so both languages stay static. English remains canonical in the page metadata and `/bn` pages are noindex.

```
src/
  app/
    [locale]/(public)/...      # public pages
    (auth)/                    # login, setup-password, reset, staff-verify
    (learner)/portal/...       # learner portal
    (admin)/admin/...          # staff portal
    api/
      payments/sslcommerz/{ipn,success,fail,cancel}/route.ts
      receipts/[id]/route.ts, certificates/[id]/route.ts
      cron/{reconcile-payments,email-outbox,purge}/route.ts
      health/route.ts
  components/{ui,public,forms,portal,admin}/
  lib/{content,i18n,format,validation}/

server/{auth,enrollment,payments,email,certificates,audit,ratelimit,settings}
/
  styles/tokens.css            # generated from content/site/brand.json
supabase/{migrations,tests,seed}/
content/                       # Part 5
docs/                          # architecture, decisions, runbooks
```

6.3 Backend: modules and endpoints

Pattern: thin entry points (server action or route handler) call a service in src/server, which calls the database through the repository or an RPC function. Every service starts with an authorization call. Files in src/server import `server-only`.

Entry point	Method	Auth	Rate limit	Purpose
Server action enrollSubmit	POST	None, Turnstile	By IP and email	Create or reuse enrollment, reserve seat, start payment or confirm free
Server action eventRegister	POST	None, Turnstile	By IP and email	Free event registration and pass
Server action assessmentSubmit	POST	None, Turnstile	By IP	Score on the server, store, email result
Server action contactSubmit	POST	None, Turnstile	By IP and email	Store inquiry, notify staff
Server action statusLookup	POST	None	Strict, by IP and reference	Return status and next step only
/api/payments/sslcommerz/ipn	POST	Hash plus server validation	Provider only	Idempotent payment confirmation
/api/payments/sslcommerz/success, fail, cancel	POST then 303 redirect	None; never trusted	Light	Re-validate server-side, redirect to a result page
/api/receipts/[id], /api/certificates/[id]	GET	Learner owns it, or staff	Standard	Signed short-lived URL after authorization

Entry point	Method	Auth	Rate limit	Purpose
Server actions (admin)	POST	Staff, permission, verified factor	Standard	Content, batches, refunds, certificates, exports
/api/cron/*	GET	Secret header	n/a	Reconcile pending payments, send queued email, purge expired data
/api/health	GET	None	Light	Uptime check without internals

6.4 Database design

Postgres with SQL migrations only. Money columns are `bigint` in poisha with a non-negative check; currency is fixed to BDT. Emails use `citext`. Public identifiers are random references, never sequence numbers.

Table	Purpose	Key columns and constraints
profiles	Activated learners and staff	id = auth user id; email unique; certificate_name; role (learner, admin, super_admin); account_state (active, disabled, archived)
courses	Reusable course	slug unique; outcomes, topics, FAQ as validated jsonb; refund policy fields; status; created_by, updated_by, published_at, archived_at
batches	Dated offering	course_id; capacity nullable (null = unlimited); list_fee_minor; enrollment window; status; certificate_template_id

Table	Purpose	Key columns and constraints
discounts	Universal or batch discount	scope; batch_id required only for batch scope; type percent or fixed; dates; status; creator; check final fee never below zero
enrollments	Application and enrollment	public_ref unique; batch_id; email; full_name; phone; consent timestamps with policy versions; status (pending, review, confirmed, expired, cancelled, completed); hold_expires_at; original_fee_minor, discount_source, discount_amount_minor, final_fee_minor snapshots; partial unique index on (batch_id, email) for non-expired rows
payment_attempts	One gateway interaction	enrollment_id; tran_id unique (30 characters at most); amount_minor; status; val_id unique when set; bank_tran_id; validated_amount_minor; risk_level; timestamps
payment_events	Append-only gateway log	tran_id; source (ipn, success, fail, cancel, validation, reconcile); redacted payload; signature_valid; received_at; no update or delete grant
receipts	Issued receipts	receipt_no from a sequence; payment_attempt_id unique; pdf_path
refund_requests	Manual refund workflow	enrollment_id; reason; status; decision fields; transfer amount, date, reference; check that processed requires all three transfer fields

Table	Purpose	Key columns and constraints
certificates	Issued certificates	enrollment_id; certificate_no unique; name snapshot; template_id; issued, reissued, revoked fields with actors; one active certificate per enrollment
certificate_templates	Artwork and field map	background path; field positions as validated jsonb
announcements, announcement_deliveries	Course or batch notices and send results	idempotency_key unique per announcement and recipient
events, event_gallery, event_registrations	Events and free registrations	unique (event_id, email); pass_no unique; capacity check inside a function
mentors, course_mentors, event_mentors	Public mentor content and links	status; photo path and alt text
reviews	Approved testimonials	consent_recorded_at and consent_evidence required before approval
hero_slides, impact_figures, site_settings	Homepage and contact content	impact_figures carry provenance and verified flag
contact_inquiries	Inquiries	status; assigned_to; internal_notes; anonymize_after
assessment_versions, assessment_questions, assessment_options, assessment_responses	Versioned bank and results	option score 0 to 4; response stores version, answers, total, percentage, label, consent flags kept separate
email_outbox	Every transactional email	template; idempotency_key unique; status; provider_message_id; retry_count; last_error
audit_events	Protected action history	actor, role, action, target, before and after, IP and user agent hash, time; append-only

Table	Purpose	Key columns and constraints
export_log	Who exported what and when (FR-ADM-004)	actor, type, filters, row_count
staff_sessions	Verified staff sessions	session_id from the JWT; method (totp or email_code); verified_at; expires_at; revoked_at
staff_email_codes	Email second factor	code_hash only; expires_at; used_at; attempts

Row-level security. Anon may read only rows with status published through read-only views. A learner may read only rows tied to their own profile, with no direct insert or update on operational tables. All staff operations run in server code with the service key behind a `withStaff(permission, fn)` wrapper, because the custom email second factor cannot raise Supabase's authenticator assurance level; the server checks the staff_sessions row on every call. pgTAP tests prove each policy.

6.5 Authentication and authorization design

Topic	Design
Account creation	Supabase signup disabled. Learners are created by the server after a validated payment or confirmed free enrollment, then receive a setup-password email. Staff are created by a Super Admin
Roles	profiles.role is the source of truth, mirrored into app_metadata only by server code. Never read a role from user_metadata, which users can edit
Learner session	Supabase session cookies through @supabase/ssr; idle expiry and absolute expiry configured in Supabase; logout revokes the session
Staff step-up	After the password step, staff verify a second factor: authenticator app (Supabase TOTP, session reaches aal2) or emailed code (custom). Either result creates a staff_sessions row bound to the JWT session_id. Staff pages and actions require that row

Topic	Design
Email code	Six digits, single use, 10-minute life (assumption), five attempts, stored hashed with a per-code salt, rate-limited by user and IP. Allowed for Admin; Super Admin must use an authenticator app (G5)
Staff timeouts	30-minute idle and 8-hour absolute (assumptions; confirm)
Role or state change	Server revokes the user's sessions and staff_sessions rows at once (NFR-SEC-04)
Password reset	Reset link goes to the verified email; a staff reset also invalidates their staff_sessions and requires the second factor again. Super Admin reset is allowed only with a second Super Admin's approval, recorded in the audit log
Factor recovery	A Super Admin removes and re-enrolls another staff member's factor through the Supabase admin API after identity checks, audited. Two Super Admin accounts must exist so one can recover the other
Authorization layer	A data access layer exposes requireLearner(), requireStaff(permission), requireOwnership(resource). Pages and actions call it; middleware only redirects for convenience and is never the sole check

6.6 Enrollment, seat and payment lifecycle

Enrollment states. `pending` (created, seat held) goes to `confirmed` on a validated payment or a confirmed free enrollment, or to `expired` when the hold lapses without payment. `review` holds risk-flagged payments, amount mismatches and paid-but-seat-lost cases until staff decide. `confirmed` goes to `completed` when an admin records completion, and a staff-approved cancellation produces `cancelled`. Whether a processed refund frees the seat is not specified in the SRS; default is no, with a staff checkbox.

Payment attempt states. `initiated`, `redirected`, then `validated`, `failed`, `cancelled`, `expired`, or `review_hold`. Failed, cancelled and expired attempts stay recorded (FR-PAY-007).

Paid enrollment flow

1. The applicant submits the universal form. The server validates input, Turnstile and consent versions.
2. RPC `start_enrollment(batch, applicant)` locks the batch row, rejects closed or full batches (BR-06), finds or creates the enrollment (G15), calculates original fee, discount and final fee from the database, stores the snapshots, and creates a seat hold of 30 minutes (assumption).
3. The server creates a payment_attempt with a new tran_id, calls the SSLCOMMERZ session API with the stored amount, the four callback URLs and the tran_id, and redirects the browser to the hosted page.
4. SSLCOMMERZ sends the IPN. The server verifies the notification hash, calls the Order Validation API, and compares tran_id, status, currency and amount with the stored attempt.
5. RPC `confirm_payment(attempt)` locks the attempt row and returns the existing result if it is already validated (idempotency). Otherwise it marks paid, confirms the enrollment if a seat is free, creates the receipt row and outbox emails, and queues account activation. All of it commits together.
6. The browser returns to the success page, which calls the same validation path and shows the stored state. If validation has not arrived it shows "pending" and polls the status for a short time.
7. A cron job reconciles attempts that stay pending beyond 15 minutes by querying the transaction status by tran_id.

Free enrollment flow. Steps 1 and 2 without a price; the seat hold stays until the emailed link is opened (G4), then `confirm_free_enrollment()` confirms and creates the account.

Capacity rule. Available seats equal capacity minus confirmed enrollments minus unexpired holds, calculated inside the locked function. A hold is also treated as expired at read time, so correctness never depends on a cron job (BR-05). `pg_cron` deletes expired holds for tidiness.

Late or conflicting payments. A validated payment after hold expiry confirms only if a seat is still free; otherwise the enrollment moves to `review`, staff are alerted, and the payment is refunded manually (G6).

6.7 Email workflows

Every email is a row in `email_outbox`, written in the same transaction as the state change. A sender attempts delivery after commit and cron retries failures with backoff, so email never blocks or reverses a payment (FR-PAY-009). The idempotency key is template plus entity id plus event, so repeated gateway callbacks cannot double-send.

Template	Trigger
assessment_result	Assessment submitted with consent
application_received, free_enrollment_confirmed	Enrollment created or free confirmed
payment_receipt, account_setup	Payment validated
password_reset	Reset requested (Supabase Auth mail sent through Resend SMTP)
event_pass	Event registration
announcement	Admin publishes with email enabled; per-recipient delivery rows
refund_status	Each refund state change
certificate_issued	Certificate generated or reissued
contact_alert	Inquiry received
staff_email_code	Staff second factor requested

Resend webhooks update delivery state. Staff can resend a failed pass or receipt from the admin.

6.8 Files and assets

Bucket	Access	Contents	Rules
public-media	Public read, server-only write	Covers, mentor photos, gallery	JPEG, PNG, WebP only; checked by file signature, not the declared type; size limit; alt text required (FR-CMS-005); EXIF removed
private-documents	No public access	Receipts, certificates	Delivered by a signed URL of 60 seconds after an ownership check
certificate-templates	Private, staff only	A4 landscape artwork	Field map validated against a schema

6.9 Performance and caching

- Public pages are static with tag-based revalidation. A publish action calls one wrapper, `lib/cache/revalidate.ts`, so a change in Next.js cache APIs touches one file.
- Public reads use the anon key under RLS or the JSON repository. They never use the service key.
- Server workloads use Supabase's connection pooler (SRS 9.2).
- Images go through next/image from the media bucket only. Budget per page: LCP image under 150 KB for mobile, JavaScript under 170 KB compressed on public pages.
- The 500-reader burst is absorbed by the CDN; transaction endpoints are load-tested separately (NFR-PER-03).

6.10 Deployment architecture

Environment	Hosting	Database	Payments	Notes
Local	Developer machine	Supabase CLI local stack	Sandbox, IPN through a tunnel	Seeded from JSON; no real credentials
Preview	Vercel preview per pull request	Staging project	Sandbox	Indexing blocked; deleted with the PR
Staging	Vercel, staging domain	Staging Supabase project	Sandbox	Mirrors production configuration; used for UAT and restore drills
Production	Vercel, lnabd.org	Production Supabase project	Live, enabled last	Protected branch, manual promotion, rollback by redeploy

Secrets are set per environment in Vercel and Supabase and never in the repository. Callback and IPN URLs are fixed per environment and registered in the matching SSLCOMMERZ panel.

6.11 Errors, logging and observability

- A typed `AppError` with a stable code. Users see a plain message and a support reference (SRS 11.3); stack traces and database errors never leave the server.

- Structured JSON logs with a request id. Redaction list: passwords, tokens, store password, full names and emails in payment logs (log ids instead), raw payloads (stored in payment_events with redaction).
- Sentry events for payment validation errors, failed jobs, email failures and unexpected exceptions (NFR-OBS-01), scrubbed in `beforeSend`.
- Alerts: any validation mismatch, any review_hold, outbox failures above threshold, cron not run in 30 minutes, error rate spike.

6.12 Backup and recovery

- Daily automated Supabase backups. Check whether the plan offers point-in-time recovery; if it does not, add an encrypted logical dump to separate storage.
- Targets (assumptions to confirm): recovery point 24 hours or better, recovery time 4 hours.
- Quarterly restore into an isolated project, verify core records and log the result (NFR-BCK-01). The first restore test happens before launch.
- Storage buckets are exported weekly. Migrations are forward-only with a written rollback note per migration.

Part 7: Security architecture and payment protection

Security is built in layers so that no single mistake, including an AI-generated one, moves money or exposes records: the browser is never trusted, the server re-checks identity, role, ownership and price on every protected action, the database enforces constraints and locks, and the payment gateway's own server-side validation decides whether money arrived. Nothing here guarantees absolute security. The plan reduces risk through controls, automated testing, monitoring and an independent penetration test before live payments.

Target standard: OWASP ASVS Level 2 for the whole application, and the OWASP Top 10 as the review checklist. Pin the ASVS and Top 10 editions at Phase 0 (verify which are current) and record them in docs/security/standard.md.

7.1 Assets and trust boundaries

Asset	Why it matters	Boundary that protects it
Payment state and gateway credentials	Direct financial loss	Server-only code, gateway validation, database functions

Asset	Why it matters	Boundary that protects it
Learner and applicant personal data	Privacy law exposure	RLS, server authorization, retention jobs
Staff and Super Admin accounts	Full control of content, refunds, exports	Passwords, second factor, staff_sessions, audit
Certificates and receipts	Fraud, reputation	Private storage, signed URLs, numbering in the database
Public content	Reputation, SEO	Draft and publish control, audit
Secrets and service keys	Total compromise	Environment isolation, scanning, rotation

7.2 Threat model

#	Threat	Example	Primary controls	Verified by
T1	Forged payment notification	Attacker posts a fake success to the IPN or success URL	Hash check, server call to the Order Validation API, tran_id, amount, currency and store checks, no trust in redirect parameters	Sandbox tests AT-01 to AT-03, forged-callback test
T2	Amount manipulation	Buyer edits the price field	Amount is computed in the database from batch and discount; the browser never sends an amount	API test with tampered body
T3	Replay or duplicate callback	Gateway retries or attacker replays	Unique val_id and tran_id; confirm_payment returns the existing result under a row lock	AT-03
T4	Seat oversell	100 simultaneous submissions for the last seat	Batch row lock inside the reserve and confirm functions	AT-04 concurrency test
T5	IDOR on receipts, certificates, refunds	Learner changes an id in the URL	Ownership checked on the server and by RLS; unguessable public references; signed URLs	AT-07

#	Threat	Example	Primary controls	Verified by
T6	Broken access control in admin	Admin calls a Super Admin action; learner calls an admin action	withStaff(permission) on every action; permission matrix as data and as tests	AT-08, matrix test over every action
T7	Credential stuffing and brute force	Login, staff code, status lookup	Rate limits, generic errors, Turnstile after failures, lockout windows	Rate-limit tests
T8	Staff account takeover	Phished staff password	Second factor, short sessions, re-auth for sensitive actions, session revocation on reset	AT-08
T9	Second-factor bypass through email	Attacker with a mailbox gets reset and code	Super Admin authenticator app only; reset of a staff password invalidates staff sessions and demands the second factor again	Auth tests
T10	SQL injection	Crafted search or filter value	Parameterized queries and RPC arguments only; no string-built SQL; input schemas	Semgrep rule, tests with payloads
T11	Stored XSS	Malicious text in mentor bio, review, inquiry	Escape by default; no raw HTML rendering; markdown rendered through a sanitizer if used at all; CSP	XSS payload tests
T12	CSRF	Cross-site form post to an admin action	Server actions with origin checks and SameSite cookies; route handlers that mutate require the session and check the Origin header	CSRF tests
T13	Malicious file upload	Script disguised as an image	Signature check, type allowlist, size limit, re-encoding, public bucket serves static types only	Upload tests

#	Threat	Example	Primary controls	Verified by
T14	Enumeration of applicants	Guessing references and emails on status lookup	Reference plus email, uniform responses and timing, strict limits, random references	AT-06
T15	Data leak through exports and logs	CSV with personal data, secrets in logs	Export permission and logging (FR-ADM-004), redaction, exports contain only needed columns	Log scan, export tests
T16	Secret exposure	Service key in a client bundle or in Git	env.ts split, server-only import, secret scanning in pre-commit and CI, bundle scan for known key patterns	CI gate
T17	Malicious or vulnerable dependency	Compromised package	Allowlist, exact pins, audit, no install scripts, review of updates	CI gate
T18	AI agent damage	Agent weakens a policy, adds a library, hardcodes a secret	Rules in Part 8, CODEOWNERS on sensitive paths, tests that assert policies, human review of migrations	Review gates
T19	Mass email abuse	Contact or assessment forms used to spam third parties	Turnstile, per-IP and per-email limits, result email only to the submitted address once per day	Rate-limit tests
T20	Gateway or provider outage	Delayed IPN, email provider down	Pending state, reconciliation job, outbox retries (SRS 10.2)	Failure-injection tests

7.3 Controls by area

Area	Controls
Authentication	Passwords of at least 12 characters, checked against common and breached lists if the plan supports it; generic login errors; setup and reset links single use and short lived; session cookies Secure, HttpOnly, SameSite Lax; idle and absolute expiry; logout revokes the server session; re-authentication before role changes, exports and financial corrections
Authorization	Permission matrix from SRS 3.2 encoded in one file, used by the server wrapper and by tests; every protected page and action checks authentication, role, account state and ownership; middleware is never the only check; no role from user-editable fields
Admin protection	Second factor, staff_sessions, separate /admin path, no admin links in public HTML, audit of publish, discount, payment, refund, certificate and staff actions (SRS 10.1), two Super Admins, access review each quarter
Input and output	Zod validation on every server entry point; output encoding by the framework; no dangerouslySetInnerHTML without sanitizer and review; upload validation as above
API	Origin checks on mutations; CORS closed (no cross-origin API); generic error bodies; rate limits per section 7.6; request size limits
Database	RLS on every table; anon and authenticated roles have no write grants on operational tables; SECURITY DEFINER functions set a fixed search_path and re-check their inputs; append-only grants on payment_events and audit_events; constraints for capacity, duplicates, money checks
Privacy	Collect only SRS fields; consent text versions stored with each record; separate marketing consent; analytics receive no personal fields (FR-OTH-009)

7.4 Payment verification design

Invariants. (1) Only server code in src/server/payments may call the gateway. (2) A payment becomes paid only inside `confirm_payment()`, and only after Order Validation succeeds with matching tran_id, amount, currency and store. (3) The success, fail and cancel URLs never mark anything paid by themselves. (4) The browser never sends or receives a price used for charging. (5) No card data, wallet credential or store password is stored or logged.

`processGatewayNotification(payload, source)` steps

1. Store the raw payload, redacted, in payment_events with source and receive time. Never fail silently.
2. Look up the attempt by tran_id. Unknown tran_id: record and return without effect.
3. If the attempt is already terminal and validated, return the existing result (idempotency).
4. Verify the notification signature (verify_sign over the fields the gateway lists, using the store password), as the gateway's documentation describes. Invalid: record, alert, stop.
5. Call the Order Validation API server-to-server with the val_id and store credentials. Network or gateway error: leave the attempt pending, return a retryable status, and let reconciliation finish it.
6. Accept only VALID or VALIDATED. Compare tran_id, amount (decimal string parsed to poisha, never floating point), currency BDT and store id with the stored values. Any mismatch: set review_hold, alert staff, stop.
7. If the gateway reports a risk flag, set review_hold and do not activate the account (FR-PAY-008).
8. Call `confirm_payment()` once; it is atomic and idempotent. Outbox emails are created inside it.
9. Return a plain acknowledgement to the gateway after the work is recorded.

Reconciliation and records. A cron job checks pending attempts older than 15 minutes through the gateway's transaction query by tran_id and runs the same function. Finance compares the gateway settlement report with the database monthly. Every step leaves payment_events and audit rows.

Failure, cancellation, timeout and refunds. Failed, cancelled and expired attempts are retained and allow a retry while the batch is open. Refunds follow the SRS workflow: request, review, approve or reject, manual transfer, then Processed only when amount, date and reference are recorded. The website never moves money.

Sandbox and live separation. One mode per environment through `SSLCZ_MODE` with its own credentials. The app refuses to start if production runs in sandbox mode, or a non-production environment runs in live mode. The live credentials exist only in the

production project.

Kill switch. `payments_enabled` in service settings, changeable only by a Super Admin, disables new payment initiation while leaving status pages and validation running.

PCI DSS. With a hosted checkout redirect, card data never reaches LNA servers, so scope is expected to be the lightest self-assessment (SAQ A) but only SSLCOMMERZ or the acquiring bank can confirm LNA's category. Obtain that in writing, and keep the obligations that remain: secure site, no card data storage, protected credentials, current software.

7.5 Security headers and Content Security Policy

Header	Value or rule
Strict-Transport-Security	max-age of one year; add includeSubDomains after every subdomain serves HTTPS; consider preload only after launch
X-Content-Type-Options	nosniff
Referrer-Policy	strict-origin-when-cross-origin
Permissions-Policy	Camera, microphone, geolocation and payment disabled
frame-ancestors (CSP)	'none'
Content-Security-Policy	default-src 'self'; script and style from self plus a per-request nonce; connect-src limited to Supabase, Sentry, analytics and Turnstile; img-src self, data and the media bucket; form-action 'self'; object-src 'none'; base-uri 'self'

Static pages and nonces. A per-request nonce forces dynamic rendering, which would defeat the CDN for 100,000 visits a month. Decision D-CSP: pages with forms or sessions (enrollment, status lookup, event registration, assessment, contact, login, portals, admin, payment results) are rendered dynamically with a strict nonce policy. Pure browsing pages stay static and use the hash-based or integrity-based mode of the pinned Next.js version if it exists; otherwise they use 'unsafe-inline' for scripts, which is accepted residual risk because those pages render no user-submitted HTML. Verify the Next.js 16 mechanism in Phase 1 and record the result in docs/decisions.

The proxy or middleware file has been renamed in recent Next.js releases: confirm the current name in the pinned version. Do not rely on it for authorization.

7.6 Rate limits and abuse controls (assumptions, tune after testing)

Endpoint	Limit
Login	5 attempts per 15 minutes per IP and email; Turnstile after 3 failures
Staff email code	5 attempts per code; 3 codes per hour per user
Status lookup	10 per hour per IP and 5 per hour per reference
Enrollment submit	5 per hour per IP and 3 per hour per email
Payment initiation	5 per hour per enrollment
Event registration	5 per hour per IP
Assessment	10 per hour per IP; one result email per address per day
Contact	3 per hour per IP and email
IPN endpoint	No user limit; signature and validation required; request size capped

All public forms use Cloudflare Turnstile. A limiter outage fails closed for payment initiation and login, and fails open with logging for read-only lookups.

7.7 Secrets and environment variables

Class	Examples	Rules
Public (NEXT_PUBLIC_)	Supabase URL and anon key, Turnstile site key, analytics id	Treated as visible to everyone; safe only because RLS and server checks exist
Server only	Supabase service key, SSLCZ store id and password, RESEND_API_KEY, Upstash tokens, CRON_SECRET, TURNSTILE_SECRET, Sentry auth token	Set per environment in Vercel; imported only from src/server; absent from client bundles

- `src/env.ts` validates both groups with Zod at startup and fails fast on missing or mismatched values (for example live credentials in preview).
- Different keys for local, staging and production. No production secret ever reaches a developer machine or Antigravity.

- gitleaks runs as a pre-commit hook and in CI. A CI step scans the built client bundle for the service key pattern.
- Rotation runbook for every secret; rotate after any suspected exposure and when a person with access leaves.
- Antigravity never receives or writes real credentials: `.env.example` documents names only.

7.8 Dependency and supply-chain security

- Lockfile committed, exact pins, frozen installs, allowlist in docs/DEPENDENCIES.md (Part 3).
- pnpm audit plus OSV scan on every pull request; Dependabot or Renovate security updates enabled; critical advisories fixed within 7 days, high within 14 (assumptions).
- GitHub Actions pinned by commit SHA; workflow permissions set to read-only by default; secrets scoped to environments.
- Branch protection on main: required checks, one human review, no force pushes. CODEOWNERS requires your review for src/server/payments, supabase/migrations, src/server/auth, .agents and AGENTS.md.
- Two-factor on every account: GitHub, Vercel, Supabase, SSLCOMMERZ panel, Resend, domain registrar, DNS and the mailbox used for them.
- Generate an SBOM per release and archive it.

7.9 Bangladesh-specific obligations

I searched for the current legal position. What I could confirm comes from news and compliance summaries, not from the official gazette, so everything below needs review by a Bangladeshi lawyer before launch.

Topic	What I found	Status	Effect on the build
Personal Data Protection Ordinance 2025 (Ordinance No. 61 of 2025)	Reported as promulgated on 6 November 2025; opt-in consent; the National Data Governance Authority enforces it; some provisions reported to start about 18 months later (around May 2027)	Needs professional confirmation	Consent records with policy versions, access and correction process, retention schedule, breach process

Topic	What I found	Status	Effect on the build
Cross-border transfer and data location	The Ordinance restricts transfers abroad without equivalent protection; one law-firm summary of the draft says confidential and restricted data must stay in Bangladesh	Needs professional confirmation, may block production	Choose Supabase region and backup location only after advice (G13)
Consumer protection and e-commerce rules	Not verified in this session	Needs professional confirmation	Likely requires seller identity, price and refund terms on the site; confirm exact fields
Payment gateway licensing and merchant onboarding	SSLCOMMERZ requires merchant onboarding and live credentials (SRS 14); regulator rules for LNA itself not verified	Needs professional confirmation	Onboarding documents and settlement reconciliation owner
VAT or tax on course fees and receipts	Not verified	Needs accountant confirmation	Receipt format and numbering (G14)
Breach notification	Not verified	Needs professional confirmation	Incident runbook includes a legal contact and decision step

Verified from official documentation this session: SSLCOMMERZ's Order Validation API requires checking amount, currency, transaction id and status server-side, and the Order Validation checkpoints list the same four checks.

7.10 Security verification strategy

1. Per phase: a short threat review against the table in 7.2, recorded in docs/security/phase-N.md.
2. Continuous: CodeQL and Semgrep, secret scanning, dependency scanning, pgTAP policy tests, authorization matrix tests.

3. Staging: OWASP ZAP baseline scan, then an authenticated scan of admin and portal paths.
4. Before live payments: independent penetration test covering authentication and second factor, IDOR, the full payment flow with forged callbacks, admin privilege escalation, file upload and export. Fix findings, then retest.
5. After launch: quarterly access review, dependency review monthly, annual penetration test, backup restore each quarter.
6. Incident response plan (Phase 14): roles, severity levels, contacts, steps for suspected payment fraud (set payments_enabled off, preserve payment_events, contact the gateway), key rotation, and a legal decision point for notification.

7.11 Security test requirements

Test	Tool	Pass condition
Forged or replayed IPN, success, fail and cancel	Vitest with sandbox fixtures, Playwright	No paid state without valid hash and validation; replay changes nothing
Tampered amount, batch or enrollment id	API tests	Server rejects or recomputes
Cross-learner access to every protected resource	Playwright, API tests	403 or 404 in every case
Role matrix over every admin action	Generated tests from the matrix file	Each role allowed or denied exactly as specified
RLS policies	pgTAP	anon and authenticated cannot read or write what they should not
Rate limits and Turnstile	Integration tests	Limits trigger; responses uniform
XSS, injection, CSRF payload set	Playwright, ZAP	No execution, no data change
Secret exposure	gitleaks, bundle scan	None found
Header and CSP check	Automated test	All headers present on every route group

Part 8: Antigravity project rules

Put the rules below in `AGENTS.md` at the repository root, with detail split into `.agents/rules/` files and long-form decisions in `docs/`. The sources I opened say Antigravity reads rules from AGENTS.md (the cross-tool format) and its own GEMINI.md, supports a workspace `.agents/rules` folder, `.agents/skills/<name>/SKILL.md` and saved workflows, and caps a rules or workflow file at 12,000 characters. That cap came from a secondary guide, so check it in your version and keep AGENTS.md well under it, as the text below is.

8.1 How context survives between sessions

Layer	File	Purpose	Changed by
Always-loaded rules	AGENTS.md	The non-negotiable rules and pointers	You only
Topic rules	.agents/rules/*.md	Security and payments, content JSON, UI and accessibility, testing, database	You only
Workflows	.agents/workflows/*.md	start-task, finish-task, security-review	You only
Decisions	docs/decisions/NNNN-*.md	One short record per architecture decision	Agent proposes, you approve
Requirements	docs/SRS.md and docs/traceability.csv	Requirement IDs and status	Agent updates status with evidence
Progress	docs/STATE.md	What is done, what is next, open questions	Agent at the end of every task

A new session needs only AGENTS.md, STATE.md and the task prompt. Never rely on chat history.

8.2 AGENTS.md (paste as written)

```
# LNA Website: agent rules (read fully before every task)

## Project
Rebuild of lnabd.org for Lead Nation Academy: public site, learner portal,
```

```
admin portal.
Offline courses only. Paid enrollment through SSLCOMMERZ hosted checkout
(BDT).
Sources of truth, in order: docs/SRS.md (requirement IDs),
docs/architecture.md,
docs/decisions/, docs/STATE.md. If a task prompt conflicts with them, STOP
and ask.

## Fixed stack
Next.js 16.x App Router, TypeScript strict, Tailwind 4, Supabase (Postgres,
Auth,
Storage), Zod, pnpm, Vitest, Playwright, Resend, Upstash, Sentry.
Versions: package.json and pnpm-lock.yaml. Node: .nvmrc.

## Rules
R1 Architecture is fixed. Do not change stack, folder layout, data model,
auth model or
   payment flow without an approved ADR in docs/decisions. If a task seems to
need it, stop.
R2 Read before write. Before editing read: this file, docs/STATE.md, the
task's SRS IDs,
   and every file you will modify. List them in your plan first.
R3 Dependencies: only packages in docs/DEPENDENCIES.md. No new packages, no
version
   changes, no "latest", no lockfile regeneration. If one is needed, stop and
ask with
   reason, alternatives, size and maintenance status.
R4 Content: no visible text, name, price, contact detail, URL or color
hardcoded in
   components. Use @/lib/content repositories, message keys via t(), and
design tokens.
   Components never import from /content directly.
R5 Money: integer poisha, never floats. Never compute a chargeable price in
the browser or
   from JSON. Use the server pricing function backed by the database.
R6 Reuse: search src/components and src/lib for an existing component or
helper before
   creating one. Extend instead of duplicating.
R7 Scope: change only files the task names or strictly requires. No drive-by
refactors,
   renames or formatting sweeps. Report other problems instead of fixing
them.
R8 Security, non-negotiable:
   - Every protected page, server action and route handler calls
requireLearner,
     requireStaff(permission) or requireOwnership on the server. Never rely
```

```
on middleware,
     hidden UI or client checks.
   - Never read a role from user_metadata. Never expose the service key,
store password or
     any secret to client code. src/server files import "server-only".
   - Validate all input with Zod on the server. Parameterized queries or RPC
only.
     No dangerouslySetInnerHTML.
   - Payment becomes paid only inside confirm_payment() after gateway server-
side
     validation. Never trust success, fail or cancel redirect data. Never
accept an amount
     from the client.
   - Never weaken or disable RLS, CSP, rate limits, Turnstile or tests to
make code pass.
   - Never write real credentials anywhere. .env.example holds names only.
R9 Database: change schema only with a new forward SQL migration in
supabase/migrations.
   Never edit an applied migration. Every table has RLS enabled and pgTAP
tests.
   No destructive statement without approval.
R10 Never delete enrollment, payment, refund, certificate or audit records.
Archive.
R11 Tests: add or update tests for the behavior you change. Run `pnpm verify`
and the
   task's extra checks before saying done. Never mark done with failing,
skipped or
   deleted tests, and never loosen an assertion to pass.
R12 Report honestly: what you ran, the results, and what you did not verify.
Say so when
   work is incomplete. Never call something complete without evidence.
R13 Accessibility: semantic HTML, labels, visible focus, keyboard operation,
24 px minimum
   targets, reduced motion. Add an axe check for each new page.
R14 Git: one task, one branch task/<id>-<slug>. Small commits:
   <type>(<scope>): <summary> [SRS IDs]. Never commit to main, never force
push.
   Do not edit AGENTS.md, .agents/ or docs/decisions unless the task says so.
R15 Demo data: new samples carry provenance class B. Never invent
accreditation,
   statistics, testimonials, partners or mentor claims.
R16 Errors and logs: throw AppError with stable codes. Never log secrets or
personal data.
   No console.log in committed code.
R17 Stop and ask when requirements conflict, the SRS is ambiguous, a rule
needs an
```

```
   exception, or tests reveal a design flaw. Never guess about money, auth or
deletion.
R18 End of every task: update docs/STATE.md (done, files changed, open
issues, next task)
   and docs/traceability.csv (status implemented or verified, with evidence).

## Commands
pnpm dev | pnpm verify (lint, typecheck, content:validate, unit, build) |
pnpm test |
pnpm test:e2e | pnpm content:validate | pnpm tokens | pnpm db:reset (local
only) | pnpm db:test

## Where things live
Architecture docs/architecture.md | Decisions docs/decisions/ | Security
.agents/rules/10-*
Content .agents/rules/20-* | UI .agents/rules/30-* | Testing
.agents/rules/40-*
Database .agents/rules/50-* | Task workflow .agents/workflows/
```

That text is about 4,900 characters. The topic files carry detail:

File	Content
.agents/rules/10-security-payments.md	Part 7 invariants, the processGatewayNotification steps, the threat table, banned patterns
.agents/rules/20-content-json.md	Part 5 directory tree, provenance rule, repository interface, validation commands
.agents/rules/30-ui-accessibility.md	Token usage, component inventory, breakpoints, WCAG checklist
.agents/rules/40-testing.md	Test pyramid, what each task must test, definition of done
.agents/rules/50-database.md	Migration conventions, RLS template, function template (SECURITY DEFINER, fixed search_path), pgTAP pattern
.agents/workflows/start-task.md	Read list, plan format, stop conditions
.agents/workflows/finish-task.md	Verification list, report format, STATE.md update
.agents/workflows/security-review.md	Per-task review against threats T1 to T20

8.3 Safeguards, and how each is enforced

Risk	Rule	Mechanical enforcement (not just a request)
Changes architecture	R1	CODEOWNERS and required review on docs/decisions, AGENTS.md, .agents
Installs libraries or shifts versions	R3	CI compares package.json to the allowlist and fails on any difference; frozen lockfile
Hardcodes content	R4	ESLint rule banning string literals in JSX under src/components except via t() and props; content:validate
Duplicates components	R6	Naming and folder lint; review checklist item
Touches unrelated modules	R7	Pull request template asks for file list against the task; reviewer diffstat check
Exposes secrets	R8	gitleaks, bundle scan, server-only imports, ESLint restricted imports
Bypasses authorization	R8	Generated permission-matrix tests; every route handler and action must call a guard, checked by a custom lint test
Insecure payment logic	R8	Payment code restricted to src/server/payments; tests for forged callbacks must exist; review by you before merge
Declares unfinished work done	R11, R12	CI is the judge, not the report; traceability status changes need evidence links
Edits applied migrations	R9	CI fails if a migration file already on main changes

8.4 Operating Antigravity safely

- Use Planning mode for anything touching auth, payments, migrations or RLS; read the plan and approve it before it edits. Fast mode is acceptable for copy, styling and isolated component work.
- Set terminal execution to require review, with an allowlist for the pnpm scripts above and a deny list for destructive deletes, force pushes, writes to environment files and unknown network calls. Check the setting names in your installed version, since the product changed at its 2.0 release.

- Use the strongest model your plan offers for planning, migrations and payment tasks; the product lists Gemini models by default and several third-party models. Model names change, so verify what you can select.
- One task per conversation. Start with the template in Part 10; end by committing and updating STATE.md.
- Do not let the agent's browser use logged-in production dashboards. Keep production credentials out of the project folder entirely.

Part 9: Step-by-step development roadmap

Build in 15 phases (P0 to P14) from an empty repository to production, in dependency order: foundation, design and content layer, public site, database, authentication, admin, enrollment, payments, learner services, remaining modules, hardening, launch. Payments are built late on purpose: they depend on the database, authentication, pricing and capacity logic, and they are the highest-risk code, so they should arrive on a stable base.

9.1 Input and decision gates

Gate	What must exist	Needed before	Owner
B0	Brand extraction record: palette, fonts, logo (Part 4.1)	P2	You
D1	Answers to G1, G2, G11 (navigation scope, URL slug, SRS version)	P0	You and LNA
D2	Answers to G4, G5, G9 (free enrollment, staff second factor, discount roles)	P7 and P9	LNA
D3	Answers to G6, G7, G14 (late payment, validation wording, receipts)	P10	LNA finance
D4	Refund policy values (G10); certificate artwork and field map	P11	LNA operations
D5	Assessment questions with scores; Bangla copy	P12	LNA
D6	Verified content, policies and legal advice on data location (G3, G13)	P14	LNA and lawyer
D7	Live SSLCOMMERZ merchant account, production domain, sender domain	P14	LNA

9.2 Order and parallelism

Can run in parallel	Must stay sequential
P2 design system and P3 content layer (after P1)	P0, then P1, then P2 and P3
P4 public site and P5 infrastructure (after P2 and P3)	P5 then P6 then P7 then P8
P12 modules (event registration, assessment, contact, analytics, Bangla) alongside P9 to P11, one at a time	P9 then P10 then P11 (each depends on the previous)
Content gathering and legal review at any time	P13 then P14

For one developer working with Antigravity, treat parallel as a choice about what to do while waiting for LNA inputs, not as a reason to open two agent sessions on the same code.

9.3 Phase definitions

Phase	Must be completed	Why it belongs here	Depends on	Next
P0 Foundation	Private repository, protections, docs set, AGENTS.md and rules, traceability file	Every later session reads these; they cost almost nothing now and prevent drift	Blueprint approved, D1	P1
P1 Scaffold and CI	Pinned Next.js app, tooling, env validation, headers, health route, CI gates	Quality gates must exist before features	P0	P2, P3
P2 Design system	Brand tokens with contrast test, base components, gallery	Every page reuses these components	P1, B0	P4
P3 Content layer	Schemas, JSON repository, validators, seed from live site, i18n skeleton	Public pages and tests need data behind a swappable interface	P1	P4

Phase	Must be completed	Why it belongs here	Depends on	Next
P4 Public site	All public pages on JSON, SEO, redirects, language switch	Delivers visible value and tests the design and content layers	P2, P3	P6
P5 Infrastructure	Staging and production projects, accounts, env vars, auth config	Database work and real email need real services	P1	P6
P6 Database	Migrations, RLS, pgTAP, seed, Supabase repository, contract tests	Everything operational depends on the schema and its policies	P3, P5	P7
P7 Authentication	Login, reset, roles, guards, staff second factor, sessions	Admin and portals need identity before any write path	P6, D2	P8
P8 Admin and CMS	Content forms, publishing, uploads, discounts, staff management, cutover to the database	Courses and batches must exist in the database before enrollment	P7	P9
P9 Enrollment	Pricing, seat holds, forms, free path, outbox, status lookup	Payment needs a priced, held enrollment	P8, D2	P10
P10 Payments	SSLCOMMERZ sandbox integration, validation, receipts, reconciliation	Highest-risk code, built on stable pricing and capacity	P9, D3	P11
P11 Learner services	Portal, announcements, refunds, certificates	Needs paid enrollments to act on	P10, D4	P12, P13
P12 Remaining modules	Events registration, assessment, contact handling, reports, analytics, Bangla	Independent of payments, so they can fill waiting time	P8 (P7 for admin parts), D5	P13

Phase	Must be completed	Why it belongs here	Depends on	Next
P13 Hardening	Full acceptance suite, accessibility, performance, security scans, restore drill	Verification across the finished system	P11, P12	P14
P14 Launch	Verified content, domain, live credentials, pen test, go-live	Production readiness and real money	P13, D6, D7	Operate

Phase	Antigravity implements	Verification	Required security checks	Complete when
P0	Nothing in the app; helps draft docs and generate traceability.csv from Appendix A	A script confirms every SRS ID appears once	Repository private, 2FA, branch protection, CODEOWNERS	Rules and docs on main
P1	App scaffold, tooling, env.ts, headers, health route, CI workflow	CI passes; headers test; smoke test	Secret scan, pinned Actions, allowlist check, CSP mechanism verified	Preview deploy works and `pnpm verify` is green
P2	Token generator, components, gallery	Contrast test, axe, screenshots at 320, 768, 1280 px	No raw HTML rendering, focus handling	Components in 4.5 base set exist and pass
P3	Schemas, repositories, validators, seed, i18n	Failing-fixture tests, contract test scaffold	Secret scan in content, restricted imports	content:validate green on seed
P4	Pages, SEO, redirects	Playwright journeys, Lighthouse budgets, axe	Drafts excluded from sitemap, no unsafe HTML	All public pages render from JSON

Phase	Antigravity implements	Verification	Required security checks	Complete when
P5	env.ts checks, health with DB ping, Sentry scrubbing, runbook	Mode-mismatch tests	Signup disabled verified, per-environment keys	Staging works end to end with demo data
P6	Migrations, policies, seed script, repository	pgTAP, contract tests, staging rehearsal	RLS on every table, grants reviewed	Both repositories pass identical tests
P7	Auth flows, guards, matrix tests, staff factors	Playwright, matrix tests, brute-force tests	T6 to T9 review by you	Both factor paths and revocation work
P8	Admin shell and forms, uploads, publish pipeline	Publish, permission and upload tests	T6, T11, T13	Staff can run all content tasks without code
P9	Pricing, RPCs, forms, outbox, status lookup	Concurrency, money and rate-limit tests	T2, T4, T14, T19	No oversell, correct fees, free path works
P10	Gateway client, callbacks, confirm function, receipts, reconcile	Full sandbox matrix, forged callback tests	T1 to T3, T20; line-by-line human review	Every case in SRS 9.1 passes in staging
P11	Portal, refunds, certificates	AT-07, AT-09, AT-10	T5, ownership tests	Learner and staff flows pass
P12	Event, assessment, contact, reports, analytics, Bangla	AT-11, AT-12, AT-14	T15, T19	Each module passes its acceptance test
P13	Test and fix work only	AT-01 to AT-15, load tests, restore drill	ZAP, authenticated scan, failure injection	All quality gates green
P14	Config, redirects, final fixes	Production smoke tests	Pen test retest clean; secrets rotated	Go decision recorded

9.4 Task list

Each task is small enough for one Antigravity conversation. A task's prerequisites are the previous task in its phase plus the phase's dependencies in 9.3; its constraints and security checks come from the phase's row in 9.3; its validation is `pnpm verify` plus the phase's verification. Part 10's template turns any row into a prompt.

Task	Objective and scope	SRS IDs	Modules	Done when
P0-T1	You create the private repo, branch protection, CODEOWNERS, pull request template	NFR-MNT-01	repo settings	Protections active
P0-T2	Create docs: SRS.md, architecture.md (Parts 3, 5, 6, 7), decisions 0001 to 0006, DEPENDENCIES.md, STATE.md, traceability.csv	all	docs/	Script finds every ID
P0-T3	Create AGENTS.md and .agents rules and workflows from Part 8	NFR-MNT-01	AGENTS.md, .agents/	Files committed
P1-T1	Scaffold Next.js with exact pinned versions and lockfile	NFR-MNT-01	root config	App builds and runs
P1-T2	ESLint, Prettier, TypeScript strict, Vitest, `pnpm verify`	NFR-MNT-01	root config	`pnpm verify` green
P1-T3	env.ts with client and server split, server-only guard, security headers, health route, mode assertions	NFR-SEC-02	src/env.ts, headers, api/health	Header and env tests pass
P1-T4	CI: verify, audit, gitleaks, dependency allowlist check, migration immutability	NFR-MNT-01	.github/workflows	Required checks on main
P1-T5	Playwright smoke test with axe harness	NFR-A11Y-01	e2e/	Smoke test in CI
P2-T0	You run Gate B0 and fill brand-extraction.md	NFR-A11Y-01	docs	Record committed
P2-T1	brand.json schema, `pnpm tokens`, tokens.css, contrast test	NFR-A11Y-01	content/site, scripts, styles	Contrast test fails on a bad pair

Task	Objective and scope	SRS IDs	Modules	Done when
P2-T2	Layout components: SiteHeader, MobileNav, SiteFooter, Container, LanguageSwitch shell	FR-PUB-001, FR-PUB-007	components/ui, components/public	Axe clean at three widths
P2-T3	Form components and all states from SRS 11.3	NFR-A11Y-01	components/forms	State tests pass
P2-T4	StatusBadge, Alert, EmptyState, ClosedNotice, DataTable, FilterBar, Pagination, ConfirmDialog	FR-ADM-001	components/ui	Keyboard tests pass
P2-T5	Dev-only gallery route showing every component	NFR-A11Y-01	app/dev	Gallery axe clean, excluded from production build
P3-T1	Zod schemas with provenance for site, home, catalog, events, mentors, reviews, assessment	FR-CMS-001	content/schema	Schema tests pass
P3-T2	Repository interfaces, JsonContentRepository, factory by domain	FR-CMS-001	lib/content	Contract suite scaffold runs
P3-T3	`pnpm content:validate`: references, slugs, alt text, secret scan, i18n parity	NFR-SEC-02	scripts	Bad fixtures fail
P3-T4	Seed JSON from the live site, all disputed fields class B; demo fixtures for every domain	FR-PUB-005	content/	Validates; no unverified claim marked verified
P3-T5	next-intl with /bn prefix, message files, canonical and noindex rules	FR-OTH-007	lib/i18n, messages	Locale tests pass
P3-T6	Demo ribbon and strict production gate	FR-PUB-005	lib/content	Strict build fails on class B

Task	Objective and scope	SRS IDs	Modules	Done when
P4-T1	Home page, eight sections in order, hero slider with pause, keyboard, swipe, reduced motion	FR-PUB-001 to 004, 006	app/(public), components	AT-15 hero checks pass
P4-T2	Courses list by Open, Upcoming, Closed; course details with batch and fee display; Enroll disabled until P9	FR-CRS-003 to 005	app/(public)/courses	Closed items show no Apply
P4-T3	About, Mentors, Mentor profile pages	FR-MEN-001 (read), FR-PUB-005	app/(public)	Pages render from JSON
P4-T4	Events list with Upcoming and Previous, event details	FR-EVT-001, 002 (read)	app/(public)/events	Past events stay available
P4-T5	Policy pages from markdown, contact page layout, 404, maintenance, expired link, service failure	FR-PUB-007	app/(public)	Footer links resolve
P4-T6	Metadata, sitemap, robots, structured data, legacy redirects from 5.1 and G1, G2	NFR-SEO-01	app, next config	Old URLs return 301
P4-T7	Public journey tests and Lighthouse budgets	NFR-PER-01, NFR-CMP-01	e2e, lhci	Budgets pass
P5-T1	You create Supabase staging and production, Vercel, Resend, Upstash, Sentry and Turnstile accounts and set variables	NFR-SEC-02	provider consoles	Variables set per environment
P5-T2	Auth config: signup off, SMTP through Resend; runbook	FR-AUT-001	supabase/config, docs	Signup attempt fails

Task	Objective and scope	SRS IDs	Modules	Done when
P5-T3	Health route with database ping, Sentry with scrubbing, mode checks	NFR-OBS-01	api/health, sentry config	Staging health green
P6-T1	Migrations: profiles and content tables with RLS and pgTAP	FR-CMS-003, 004	supabase/migrations	pgTAP green
P6-T2	Migrations: enrollments, discounts, payment attempts and events, receipts	FR-PAY-003, 006	supabase/migrations	Constraints tested
P6-T3	Migrations: refunds, certificates, templates, announcements, registrations, inquiries, assessment, outbox, audit, export_log, staff tables	FR-RFC, FR-LRN, FR-ADM	supabase/migrations	Append-only grants tested
P6-T4	Seed script, SupabaseContentRepository, contract tests on both repositories	FR-CMS-001	scripts, lib/content	Identical results
P6-T5	Staging migration rehearsal and rollback notes	NFR-MNT-01	docs	Rehearsal logged
P7-T1	SSR clients, login, logout, sessions, generic errors, rate limit and Turnstile	FR-AUT-002, NFR-SEC-03	server/auth, app/(auth)	Brute-force test passes
P7-T2	Profiles, role mapping, guards, permission matrix file with generated tests	FR-ADM-002, NFR-SEC-01	server/auth	Matrix tests pass
P7-T3	Setup password, reset, expired link pages	FR-AUT-007	app/(auth)	Single-use links verified
P7-T4	Staff TOTP enroll and verify with staff_sessions	FR-AUT-004	server/auth	aal2 path works
P7-T5	Staff email code: hashed, single use, short lived, limited	FR-AUT-004, 005	server/auth	Inspection and tests pass

Task	Objective and scope	SRS IDs	Modules	Done when
P7-T6	Revocation on role or state change, access denied, auth audit events	FR-AUT-003, NFR-SEC-04	server/auth, audit	Revoked session fails at once
P8-T1	Admin shell, permission-gated menu, empty states	FR-ADM-001, 002	app/(admin)	Menu hides and server denies
P8-T2	Courses and batches editor with draft, preview, publish, archive, audit	FR-CMS-001 to 004, FR-CRS-001, 002, 007	app/(admin), server	AT-13 passes
P8-T3	Image upload pipeline with signature check and alt text	FR-CMS-005	server, storage	Bad files rejected
P8-T4	Events, mentors, reviews with consent, hero slides, impact figures, contact settings forms	FR-EVT-001, FR-MEN-001, FR-REV-001, FR-PUB-003	app/(admin)	Each can be edited without code
P8-T5	Discounts: universal for Super Admin, batch for Admin, no stacking, non-negative result	FR-PAY-001, 002	app/(admin), server	Rules tested
P8-T6	Staff management and service settings for Super Admin	FR-ADM-006, BR-09	app/(admin)	Admin denied
P8-T7	Revalidation wrapper and per-domain cutover to Supabase	NFR-PER-02, FR-CRS-004	lib/cache, lib/content	Publish updates public pages
P9-T1	Pricing function with discounts and snapshots, money tests	FR-PAY-002, 003, BR-04	server/enrollment, supabase	No float use, edge cases pass

Task	Objective and scope	SRS IDs	Modules	Done when
P9-T2	start_enrollment, seat hold, expiry at read time, capacity tests	FR-CRS-006, BR-05, 06	supabase, server	AT-04 concurrency passes
P9-T3	Enrollment form and fee review, consent versions, duplicate handling	FR-ENR-001 to 003	app/(public)/enroll	Entries survive errors
P9-T4	Email outbox, Resend sender, retries, first templates	FR-PAY-009 (base), NFR-AVL-01	server/email	Failure keeps state
P9-T5	Free enrollment confirmation and account activation	FR-ENR-005, 006, BR-01	server	AT-05 passes
P9-T6	Status lookup with limits and uniform answers	FR-ENR-004, BR-08	server, app	AT-06 passes
P10-T1	SSLCOMMERZ client, mode assertions, fixtures	FR-PAY-004	server/payments	Fixture tests pass
P10-T2	Payment attempt, redirect, success, fail, cancel handlers	FR-PAY-004, 007	api/payments	Redirects never mark paid
P10-T3	processGatewayNotification and confirm_payment, idempotency	FR-PAY-005, 006, 008	server/payments, supabase	AT-03 passes
P10-T4	Receipt PDF, email, account activation action	FR-PAY-009	server	Email failure keeps payment
P10-T5	Reconciliation cron, review hold views, kill switch	FR-PAY-008	api/cron, app/(admin)	Late payment handled
P10-T6	Full sandbox matrix and forged callback tests	SRS 9.1, AT-01 to 03	e2e, tests	All cases pass in staging
P11-T1	Learner portal shell, overview, my courses, course detail	FR-LRN-001, 002, 006	app/(learner)	Only own data visible

Task	Objective and scope	SRS IDs	Modules	Done when
P11-T2	Receipt download, profile edit, correction request	FR-LRN-003, FR-AUT-006	server, app	AT-07 passes
P11-T3	Announcements with email and duplicate protection	FR-LRN-004, 005	server, app/(admin)	No double send
P11-T4	Refund request, staff workflow, emails, Processed rule	FR-RFC-001 to 004, BR-11	server, app	AT-09 passes
P11-T5	Completion marking, templates, certificate PDF, numbering	FR-RFC-005 to 007, BR-12, 13	server/certificates	AT-10 first half
P11-T6	Reissue, revoke, learner download, certificate email	FR-RFC-008	server, app	Revoked unavailable to learner
P12-T1	Event registration, pass, resend	FR-EVT-003, 004	server, app	AT-11 passes
P12-T2	Assessment engine, UI, result email	FR-OTH-001 to 005	server, app	AT-12 passes
P12-T3	Contact inquiries and staff handling	FR-OTH-006, FR-ADM-005	server, app	Status and notes saved
P12-T4	Overview, reports, CSV exports with export log	FR-OTH-008, FR-ADM-003, 004	app/(admin)	Exports logged
P12-T5	Analytics and consent banner	FR-OTH-009, NFR-PRI-01	components, config	AT-14 passes
P12-T6	Complete Bangla labels and language tests	FR-OTH-007	messages	Parity test passes

Task	Objective and scope	SRS IDs	Modules	Done when
P13-T1	AT-01 to AT-15 in CI	all AT	e2e	Suite green
P13-T2	Accessibility audit and fixes	NFR-A11Y-01	components	Audit report signed
P13-T3	Performance and load tests, budgets	NFR-PER-01 to 03	k6, lhci	Targets met
P13-T4	ZAP, authenticated scan, failure injection, dependency review	NFR-SEC, NFR-AVL-01	CI	No open high findings
P13-T5	Backup restore drill, monitoring alerts, runbooks	NFR-BCK-01, NFR-OBS-01	docs	Restore logged
P14-T1	Convert content to class A with evidence; strict build passes	FR-PUB-005	content/	Strict build green
P14-T2	Domain, DNS, sender domain, redirects, SEO checks	NFR-SEO-01	config	Checks pass
P14-T3	Production services, live merchant credentials, live callbacks, secret rotation	FR-PAY-004	provider consoles	Live mode only in production
P14-T4	Independent penetration test, fixes, retest	NFR-SEC	external	No open high findings
P14-T5	Go decision, launch, smoke tests, hypercare	SRS 12.3	all	Record signed by owner

9.5 Checkpoints and Git practice

- Branch per task, pull request per task, squash merge to main after required checks and your review.
- Tag `phase-N-complete` when a phase's completion row is satisfied; the tag is a restore point.
- Reviews you must do yourself: every migration, everything in src/server/payments and src/server/auth, AGENTS.md and .agents, dependency files.

- A task is never started with another task's branch unmerged.

9.6 Demo data versus production integration

Area	Until P14	At launch
Payments	Sandbox credentials in local, preview and staging only	Live credentials in production only; `payments_enabled` turned on last by a Super Admin after a supervised live test
Content	Class B demo seeded from the live site	Class A with evidence; strict build enforced
Email	Test sender, staging domain, allowlisted recipients	Verified production domain
Accounts	Test learners and staff with fake data	Real staff created by Super Admin; test data purged
Database	Staging project with seed data	Production project, empty except content and settings
Analytics	Disabled	Enabled after consent wording is approved

Part 10: Antigravity implementation prompts

Paste the template into a fresh Antigravity conversation for every task, fill the angle-bracket fields from the task table in 9.4, and keep AGENTS.md as the standing context. Below: the template, the two prompts that start the project, and two worked examples for the riskiest tasks. Every other task prompt is the template filled from its row in 9.4, the phase's security checks in 9.3, and the relevant Part 5 to 7 sections.

10.1 Reusable task template

```
TASK <ID>: <title>          BRANCH: task/<id>-<slug>          MODE: <Planning
| Fast>

READ FIRST, IN ORDER (list what you read in your plan before editing
anything)
1. AGENTS.md
2. docs/STATE.md
3. <.agents/rules files that apply>
4. SRS requirement IDs: <IDs>   (find them in docs/SRS.md)
5. Existing files you will change: <paths>   (read them fully; search for
```

```
existing components or helpers first)

OBJECTIVE
<one paragraph: the outcome, not the method>

PREREQUISITES (verify each; if one is false, stop and report)
- <previous task merged, tables exist, variable set, and so on>

IN SCOPE
- <specific behaviors and files>
OUT OF SCOPE (do not touch)
- <named modules, plus: anything not listed above>

TECHNICAL CONSTRAINTS
- Use only dependencies in docs/DEPENDENCIES.md. No new packages or version
changes.
- <specific patterns, function names, money in poisha, content via
repositories, and so on>

SECURITY REQUIREMENTS
- <from the phase row in 9.3 and the threats that apply>

DELIVERABLES
- <files, migrations, tests, docs>

TESTS TO WRITE
- <unit, integration, e2e, pgTAP cases, including at least one failure case
per rule>

VALIDATION (run, then paste the results)
- pnpm verify
- <task-specific commands>

ACCEPTANCE CRITERIA
- <observable, testable statements>

STOP AND ASK IF
- A requirement conflicts or is ambiguous, a rule needs an exception, a
dependency is needed,
  or you would have to change files outside scope.

REPORT FORMAT (end of task)
1. Summary in three lines
2. Files changed (list)
3. Commands run and their results
4. Tests added
```

```
5. Not verified, risks, assumptions
6. Requirement IDs touched and proposed traceability status, with evidence
7. Suggested next task and any decision I must make
Then update docs/STATE.md and docs/traceability.csv, commit on the task
branch, and do not merge.
```

10.2 Starting prompt A: traceability file (P0-T2, Fast mode)

You place AGENTS.md, .agents/ and the docs skeleton yourself from Part 8, then ask Antigravity to generate the traceability file.

```
TASK P0-T2a: Generate docs/traceability.csv          BRANCH: task/p0-t2a-
traceability          MODE: Fast

READ FIRST: AGENTS.md, docs/SRS.md (plain-text copy of LNA-SRS-001).

OBJECTIVE
Create docs/traceability.csv with exactly one row for every requirement ID in
docs/SRS.md:
all FR-*, NFR-*, BR-*, AT-* and UC-* identifiers.
Columns: id, type, text_summary (max 20 words, your own words), priority (M,
G or F, empty for
BR, AT, UC), phase (P0 to P14), modules, acceptance_test, status, evidence.
Use my phase mapping below. Set status to "not_started" for every row and
leave evidence empty.

PHASE MAPPING
<paste the Appendix A table from the blueprint>

OUT OF SCOPE: any file other than docs/traceability.csv and docs/STATE.md.

VALIDATION
Write scripts/check-traceability.mjs (no new dependencies) that extracts
every requirement ID
by regular expression from docs/SRS.md and fails if any is missing from or
duplicated in the CSV.
Run it and paste the output. Expected counts: 77 FR, 15 NFR, 18 BR, 15 AT, 4
UC.

STOP AND ASK IF an ID in the SRS is ambiguous or the counts differ.
```

10.3 Starting prompt B: scaffold (P1-T1, Planning mode)

```
TASK P1-T1: Scaffold the Next.js application with exact pinned versions
BRANCH: task/p1-t1-scaffold          MODE: Planning

READ FIRST: AGENTS.md, docs/STATE.md, docs/architecture.md (sections on stack
and folders),
docs/DEPENDENCIES.md.

OBJECTIVE
Create the empty application skeleton so every later task builds on pinned,
verified versions.

STEP 1 (stop after this step and wait for my approval)
For every package in the "Initial allowlist" of docs/DEPENDENCIES.md, run
`pnpm view <pkg> version`
and open the package's official documentation page. Produce a table: package,
resolved version,
peer dependency requirements, supported Node versions, official source URL.
Flag any package whose
peer requirements conflict with another, and any whose latest version is a
pre-release.
Do not install anything yet. Do not take versions from memory or from third-
party blog posts.

STEP 2 (only after I reply "approved")
- Create .nvmrc and engines with the Node version I approve.
- Create the app with the approved exact versions (no ^ or ~; .npmrc save-
exact=true).
- TypeScript strict. Tailwind 4 with CSS-first configuration and no
tailwind.config.js.
- Folder layout exactly as in docs/architecture.md. Placeholder home page
only.
- package.json scripts: dev, build, start, lint, typecheck, test, verify.
- Commit the lockfile. Record the final versions in docs/DEPENDENCIES.md.

OUT OF SCOPE: components, content, Supabase, auth, CI, Playwright (later
tasks).

VALIDATION: pnpm install --frozen-lockfile, pnpm verify, pnpm build. Paste
results.

ACCEPTANCE: the app starts; verify passes; no dependency outside the
allowlist; no ^ or ~ ranges;
lockfile committed.
```

```
STOP AND ASK IF peer dependencies conflict, a package needs an install
script, or Next.js 16 needs
a configuration not described in docs/architecture.md.
```

10.4 Worked example: seat holds (P9-T2, Planning mode)

```
TASK P9-T2: Seat holds and capacity          BRANCH: task/p9-t2-seat-holds
MODE: Planning

READ FIRST: AGENTS.md, docs/STATE.md, .agents/rules/50-database.md,
.agents/rules/10-security-payments.md,
SRS IDs FR-CRS-006, BR-05, BR-06, AT-04, docs/architecture.md section 6.6,
and the migrations for
batches and enrollments.

OBJECTIVE
A limited batch reserves a seat during checkout, releases an expired hold,
and can never confirm
more learners than its capacity, even under concurrent requests.

PREREQUISITES: P9-T1 merged (pricing function exists); tables batches and
enrollments exist.

IN SCOPE
- New migration: functions available_seats(batch_id), start_enrollment(...),
release_expired_holds().
- start_enrollment locks the batch row (SELECT ... FOR UPDATE), rejects
closed or full batches,
  reuses an existing pending enrollment for the same email and batch,
snapshots fees from the
  pricing function, and sets hold_expires_at = now() + hold duration from
service settings.
- available_seats counts confirmed enrollments plus unexpired holds. Expired
holds count as free
  at read time without any job.
- pg_cron schedule for release_expired_holds as housekeeping only.
- Server wrapper src/server/enrollment/start.ts that calls the RPC and maps
errors to AppError codes.
OUT OF SCOPE: payment gateway code, UI, emails.

SECURITY: RPC executable only by the server role; anon and authenticated
cannot call it; SECURITY
DEFINER with fixed search_path; validate every argument inside the function;
no capacity or price
value is accepted from the client.
```

```
TESTS
- pgTAP: capacity 5, 5 holds succeed, the sixth fails; expired hold frees a
seat; duplicate email
  reuses the enrollment; closed batch rejected; unlimited batch (capacity
null) accepted.
- Concurrency test (Vitest, 50 parallel calls against the local database,
capacity 5): exactly
  5 succeed and 45 receive the "batch full" code. Run it 10 times.

ACCEPTANCE: all tests pass ten times in a row; no test depends on a cron job;
rollback note written.

STOP AND ASK IF the hold duration setting is missing or the enrollment status
values differ from
architecture.md section 6.6.
```

10.5 Worked example: payment notification handler (P10-T3, Planning mode)

```
TASK P10-T3: processGatewayNotification and confirm_payment
BRANCH: task/p10-t3-confirm-payment          MODE: Planning

READ FIRST: AGENTS.md, docs/STATE.md, .agents/rules/10-security-payments.md,
docs/architecture.md
sections 6.6 and 7.4 (the nine steps), SRS IDs FR-PAY-005, FR-PAY-006, FR-
PAY-008, BR-01, BR-02,
AT-03, SRS section 9.1, and the SSLCOMMERZ client from P10-T1 and its
fixtures.

OBJECTIVE
Only a gateway notification that passes hash verification and server-side
Order Validation, and that
matches the stored tran_id, amount, currency and store, can mark a payment
paid. Repeated or delayed
notifications change nothing.

PREREQUISITES: P10-T1 and P10-T2 merged; payment_attempts, payment_events and
receipts tables exist.

IN SCOPE
- src/server/payments/process-notification.ts implementing the nine steps in
architecture.md 7.4.
- Migration for confirm_payment(attempt_id, val_id, bank_tran_id,
validated_amount_minor, risk_level):
  locks the attempt row; returns the existing result if already validated;
marks paid; confirms the
```

```
  enrollment when a seat is free, otherwise sets status review; writes the
receipt row and outbox
  rows in the same transaction.
- IPN route handler calls the same function as the success route. Neither
trusts query or body
  values other than val_id and tran_id.
- Amount comparison parses the gateway's decimal string to poisha with an
exact decimal parser;
  never use Number or parseFloat.
OUT OF SCOPE: receipt PDF, activation email content, reconciliation, admin
views.

SECURITY: line-by-line review by me before merge. No logging of store
password, full payload or
personal data; log ids and outcomes only. Fail closed: any validation
uncertainty leaves the attempt
pending and creates no paid state.

TESTS (using recorded sandbox fixtures)
- valid notification confirms once; the second identical notification returns
the same result and
  creates no second receipt, account action or email row
- invalid hash; unknown tran_id; validation API error; status FAILED,
CANCELLED, EXPIRED
- amount mismatch; currency mismatch; store id mismatch -> review_hold and an
alert row
- risk_level 1 -> review_hold and no activation
- late payment after hold expiry: seat free -> confirmed; batch full ->
review
- success redirect with forged parameters and no valid validation -> not paid

ACCEPTANCE: every test passes; a grep test proves no other code path writes
status "validated";
AT-03 passes in staging with sandbox credentials.

STOP AND ASK IF the fixture fields differ from the gateway documentation, or
the hash fields list
in the documentation is unclear.
```

Part 11: Testing and deployment strategy

Automated tests judge completion, not Antigravity's report: every pull request runs static checks, unit, component, database and smoke tests; staging runs the full acceptance suite, accessibility, performance and security scans; production is promoted manually

after a checklist. Thresholds marked as assumptions are starting points to tune after the first measurements.

11.1 Test types and tools

Type	Tool	Covers	Runs
Static	TypeScript strict, ESLint with a11y rules, Prettier	Types, banned patterns (any, raw HTML, hardcoded strings, restricted imports)	Every commit and pull request
Unit	Vitest	Pricing, money parsing, scoring and labels, reference generation, permission matrix, content schemas	Pull request
Component	Vitest with Testing Library	Forms, hero slider controls, tables, dialogs, keyboard behavior	Pull request
Database	pgTAP through the Supabase CLI	RLS, constraints, functions, append-only grants, capacity and idempotency	Pull request
Integration and API	Vitest against the local stack	Server actions, route handlers, repository contract tests, rate limits	Pull request
End to end	Playwright at 320, 768 and 1280 px, Chromium, Firefox and WebKit	AT-01 to AT-15 and public journeys	Smoke on pull request, full on main and nightly
Authentication and authorization	Playwright and generated matrix tests	Roles, ownership, second factor, session revocation	Pull request

Type	Tool	Covers	Runs
Payment sandbox	Playwright plus fixtures against SSLCOMMERZ sandbox	Success, failure, cancellation, expiry, duplicate, delayed, mismatch, risk flag, forged callbacks	Staging, nightly
Accessibility	axe-core in Playwright, then manual audit	WCAG 2.2 AA (automation finds a minority of issues)	Pull request, and a manual audit in P13
Responsive	Playwright viewport projects	320 to 599, 600 to 1023, 1024 and up	Pull request
Performance	Lighthouse CI, k6	Web Vitals proxies, bundle budgets, transaction endpoint load	Lighthouse on main, k6 before release
Security	gitleaks, pnpm audit, OSV, CodeQL, Semgrep, OWASP ZAP	Secrets, dependencies, code patterns, running site	Pull request and nightly; ZAP on staging
Regression	The suites above plus a tagged bug-fix test per defect	Every fixed defect gets a test	Pull request
Backup and restore	Scripted drill	Restore into an isolated project and verify records	Before launch, then quarterly
Production smoke	Playwright against production, read-only plus one controlled enrollment path	Home, course, status lookup, login, health	After every deploy

11.2 Quality gates

Gate	Threshold	Blocks
Type and lint	Zero errors, zero warnings	Merge
Unit and component tests	All pass; line and branch coverage at least 90 percent in src/server/payments, src/server/auth and pricing; at least 70 percent elsewhere (assumption)	Merge

Gate	Threshold	Blocks
Database tests	All pass; every table has RLS tests	Merge
Content validation	Passes; strict mode passes before production	Merge, release
Dependencies	No high or critical advisory; allowlist check passes	Merge
Secrets	Zero findings	Merge
Accessibility	Zero serious or critical axe violations on core journeys	Merge to main
Web Vitals targets	LCP 2.5 s or better, INP 200 ms or better, CLS 0.1 or better at the 75th percentile (SRS NFR-PER-01); lab budgets: LCP 2.5 s, total blocking time 200 ms, CLS 0.1 on mobile emulation (assumption)	Release
Bundle budget	JavaScript under 170 KB compressed on public pages (assumption)	Merge to main
Load	Public pages: 500 concurrent readers with p95 under 800 ms and error rate under 1 percent. Enrollment, event and payment initiation: tested at a rate agreed with LNA (start with 30 concurrent) with no oversell and no duplicate record (assumptions)	Release
Security scan	ZAP baseline has no medium or higher open finding; pen test has no open high finding	Production launch
Acceptance	AT-01 to AT-15 pass on staging	Production launch

11.3 Acceptance tests to automation

AT	Journey	Automated by	Built in
AT-01	Paid enrollment	Playwright with sandbox, DB assertions on single receipt and account	P10
AT-02	Payment failure	Playwright with sandbox failure and cancel	P10

AT	Journey	Automated by	Built in
AT-03	Duplicate notification	Vitest replay, database counts	P10
AT-04	Capacity	Concurrency test, pgTAP	P9
AT-05	Free enrollment	Playwright, assertion that no gateway call occurred	P9
AT-06	Applicant status	Playwright, rate-limit and enumeration checks	P9
AT-07	Learner authorization	Playwright and API tests across two learners	P11
AT-08	Staff security	Playwright both factor paths, matrix tests	P7
AT-09	Refund	Playwright through staff and learner views, email outbox assertions	P11
AT-10	Certificate	Playwright and PDF text extraction check	P11
AT-11	Event	Playwright capacity, duplicate and resend	P12
AT-12	Assessment	Unit scoring for all labels plus Playwright, no auth user created	P12
AT-13	Publishing	Playwright draft, publish, archive with cache check	P8
AT-14	Analytics privacy	Playwright network inspection of analytics payloads	P12
AT-15	Accessibility	axe plus manual keyboard, zoom, contrast and reduced motion audit	P13

11.4 CI/CD pipeline

1. **Pull request:** install with frozen lockfile, lint, typecheck, unit and component tests, content:validate, start the local Supabase stack and run pgTAP and integration tests, build, Playwright smoke with axe, gitleaks, pnpm audit and OSV, CodeQL and Semgrep, dependency allowlist check, check that no applied migration changed, client bundle secret scan. Vercel creates a preview deployment on the staging database.
2. **Merge to main:** automatic deploy to staging, then full Playwright suite, Lighthouse CI and the sandbox payment matrix. A nightly job repeats the full suite, ZAP baseline and dependency review.

3. **Release:** a manual approval promotes the exact staging build to production. Before running migrations, confirm a fresh backup. Migrations follow expand then contract: the new schema works with the previous app version, so an application rollback never meets an incompatible database.
4. **After deploy:** production smoke tests, then watch payment validation errors, outbox failures, error rate and cron health for 48 hours.
5. **Rollback:** redeploy the previous Vercel build; database changes are forward-only with a written compensating step per migration.

11.5 Staging and user acceptance

LNA staff run the acceptance journeys on staging with the SSLCOMMERZ sandbox test cards and wallets listed in the sandbox payment page, using seeded demo batches. Each journey records tester, date, result and screenshots. Business owner sign-off follows SRS 12.3: implementation, review evidence, a passing acceptance test, related content and procedures, no blocking defect, owner approval.

11.6 Production readiness checklist

Area	Item	Owner
Security	Pen test findings fixed and retested; secrets rotated; two-factor on every provider account; kill switch tested; ZAP clean	You
Payments	Live merchant account approved; live callback and IPN URLs registered; reconciliation owner named; supervised live test then manual refund recorded	LNA finance
Data	Region decision documented (G13); backup running; restore drill logged; retention and purge jobs enabled	You and legal
Legal	Privacy, Terms and Refund Policy approved and linked; consent text versions set; data subject request procedure written	LNA and lawyer
Content	All rendered records class A; strict build green; mentor consent on file	LNA
Operations	Monitoring, alerts, runbooks, support mailbox, incident contacts	You
Email	SPF, DKIM, DMARC set; test delivery to Gmail and local providers	You

Area	Item	Owner
Search	Redirect map tested; sitemap submitted; metadata and canonical checked	You
Quality	AT-01 to AT-15 green; manual accessibility audit signed; load test passed	You
Rollback	Rollback rehearsed on staging	You

11.7 Moving from demo to production

Step	What changes	JSON only	Database migration	Code change	Console or provider
Replace sample content with verified information	Records and provenance	Yes before cutover, admin forms after	No	No	No
Update names, contact details, navigation, footer, SEO	Configuration	Yes (site files)	No	No	No
Connect real backend data	Per-domain content source switch	No	Already applied in P6	No	Environment variable
Production authentication	Signup off, SMTP, staff accounts, two Super Admins	No	No	No	Supabase and Resend
Move sandbox to live payments	Credentials, mode, live callback URLs	No	No	No	Vercel variables, SSLCOMMERZ panel
Validate payment integration before activation	Supervised live transaction, refund, reconciliation	No	No	Only if a defect appears	Gateway panel
Configure production domain	DNS, HTTPS, canonical domain, Vercel domain	No	No	No	DNS, Vercel
Configure email	Sender domain, templates reviewed	Templates in code	No	Template edits only	DNS, Resend
Migrate content from the current site	Course, mentors, policies, media into the admin or seed	Yes	No	No	No

Step	What changes	JSON only	Database migration	Code change	Console or provider
Preserve URLs and SEO	301 map for old paths, keep the IRGP slug, metadata parity, sitemap	Redirect map in config	No	No	Search Console
Privacy, security and operations readiness	Section 11.6	No	No	Fixes only	Various

A field added to content, a new rule, a new role or a new workflow always needs a code change, and a new column needs a migration. Routine wording, images, courses and batches never do.